

RAPPORT DE STAGE

DEVELOPPEMENT D'UNE APPLICATION WEB DE GESTION D'INVENTAIRE

Centre CSZIAM — INRH

Élaboré par :

Hiba EL KHAYAT

Encadrée par :

Mme.Nadia Bouhriz,M.Mustapha Wafik ET M.Balmani
Chawki

Période de stage : du 13 juillet au 13 septembre 2026

Année universitaire :2025/2026

Dédicace

Je dédie ce travail à mes parents,

pour leur soutien et leurs encouragements tout au long de mes études.

Je le dédie aussi à ma famille et à mes amis,

qui m'ont accompagnée et motivée pendant ce stage.

Remerciements

Je tiens à remercier en premier lieu M. Mustapha Wafik, mon encadrant au Centre CSZIAM, pour le temps qu'il m'a consacré, ses conseils techniques et sa disponibilité tout au long du stage. Ses remarques m'ont beaucoup aidé à avancer, notamment dans les moments de blocage.

Je remercie également M. Mohamed Naoufal Tamsouri, ainsi que l'ensemble du personnel de l'INRH et du Centre CSZIAM, pour leur accueil et pour m'avoir permis de travailler dans de bonnes conditions pendant ces deux mois.

Je remercie également mes professeurs de l'EMSI, qui m'ont donné les bases nécessaires pour aborder ce stage sans être complètement perdue face aux technologies utilisées.

Enfin, merci à ma famille et à mes amis pour leur soutien pendant cette période, et à toutes les personnes qui ont, d'une manière ou d'une autre, contribué à ce travail.

Résumé

Ce rapport présente le travail réalisé dans le cadre d'un stage de deux mois effectués à l'INRH, pour le Centre CSZIAM. L'objectif du stage était de développer une application web permettant de centraliser et de gérer le patrimoine matériel du centre : les articles, les catégories, les laboratoires, les services organisationnels, les utilisateurs, les mouvements du matériel, les campagnes d'inventaire annuel, ainsi que le suivi statistique du patrimoine.

Le backend de l'application a été développé en Java 21 avec Spring Boot, organisé en couches (contrôleurs, services, repositories, entités, DTO) et connecté à une base de données MySQL avec Spring Data JPA. La sécurité repose sur Spring Security avec une authentification par JWT. Le frontend a été développé avec Next.js, React, TypeScript et Tailwind CSS. Une partie importante du travail a porté sur la traçabilité des mouvements du matériel et sur des règles métier appliquées côté serveur.

Le projet comprend également une fonctionnalité de génération de rapport d'inventaire assistée par intelligence artificielle, à l'aide d'un modèle local exécuté avec Ollama, ainsi qu'une génération de document PDF avec la bibliothèque OpenPDF. Les API ont été documentées et testées avec Swagger/OpenAPI et Postman. Ce rapport revient sur la démarche suivie, de l'analyse jusqu'aux tests, ainsi que sur les difficultés rencontrées, les solutions apportées, et les perspectives d'amélioration du projet.

Mots-clés:

Java 21, Spring Boot, API REST, MySQL, Spring Data JPA, DTO, JWT, Next.js, React, TypeScript, Tailwind CSS, Swagger/OpenAPI, Postman, Ollama, OpenPDF, gestion de patrimoine, gestion d'inventaire, traçabilité.

Abstract

This report presents the work carried out during a two-month internship at INRH, for the CSZIAM Center. The goal of the internship was to develop a web application to centralize and manage the center's material assets: items, categories, laboratories, organizational departments, users, equipment movements, annual inventory campaigns, and statistical monitoring of the assets.

The application backend was developed in Java 21 with Spring Boot, organized in layers (controllers, services, repositories, entities, DTOs) and connected to a MySQL database through Spring Data JPA. Security relies on Spring Security with JWT-based authentication. The frontend was built with Next.js, React, TypeScript and Tailwind CSS. A significant part of the work focused on the traceability of equipment movements and on business rules enforced on the server side.

The project also includes an AI-assisted inventory report generation feature, using a local model run with Ollama, as well as PDF document generation with the OpenPDF library. APIs were documented and tested with Swagger/OpenAPI and Postman. This report goes over the approach followed, from analysis to testing, as well as the difficulties encountered, the solutions applied, and the perspectives for improvement.

Keywords:

Java 21, Spring Boot, REST API, MySQL, Spring Data JPA, DTO, JWT, Next.js, React, TypeScript, Tailwind CSS, Swagger/OpenAPI, Postman, Ollama, OpenPDF, asset management, inventory management, traceability.

Table des matières

Introduction générale	1
Chapitre 1 — Présentation de l'organisme et du contexte.....	2
1.1 Introduction.....	2
1.2 Présentation de l'organisme d'accueil.....	2
1.3 Présentation du Centre CSZIAM	3
1.4 Contexte et déroulement du stage	3
1.5 Sujet du stage	5
1.6 Problématique	5
1.7 Objectifs du projet.....	5
1.8 Conclusion	6
Chapitre 2 — Spécification des besoins	7
2.1 Introduction.....	7
2.2 Identification des acteurs	7
Administrateur	7
Responsable de laboratoire ou de service	7
Agent d'inventaire	7
2.3 Besoins fonctionnels	7
2.4 Besoins non fonctionnels	8
2.5 Règles métier	8
Règle relative aux responsables	8
Règle relative aux états du matériel	9
2.6 Synthèse des exigences par acteur	9
2.7 Cas d'utilisation.....	9
2.8 Conclusion	10
Chapitre 3 — Conception du système.....	11
3.1 Introduction.....	11
3.2 Architecture générale	11
3.3 Architecture backend	12
3.4 Architecture frontend.....	12
3.5 Pourquoi ces choix technologiques.....	12
3.6 Modélisation des données	13
Entité Article.....	13
Entité Mouvement.....	13
Entités CampagneInventaire et Inventaire	13
Relations entre les entités.....	14

3.7 Diagramme de classes.....	14
3.8 Diagrammes de séquence.....	14
3.9 Gestion des rôles.....	16
3.10 Conclusion	16
Chapitre 4 — Réalisation du système	17
4.1 Introduction.....	17
4.2 Environnement et technologies.....	17
4.3 Organisation du backend.....	17
4.4 Gestion des utilisateurs	17
4.5 Gestion des articles et du matériel	18
4.6 Gestion des catégories, laboratoires et services organisationnels	18
4.7 Validation des données	18
4.8 Gestion des mouvements	18
4.9 Traçabilité des mouvements.....	19
4.10 Règles métier liées aux états du matériel	19
4.11 Gestion des campagnes d'inventaire et du pointage.....	19
4.12 Statistiques et tableau de bord.....	19
4.13 Sécurité et authentification JWT.....	20
4.14 Documentation et test des API avec Swagger/OpenAPI	20
4.15 Génération du rapport d'inventaire assisté par intelligence artificielle	23
4.16 Génération du document PDF.....	25
4.17 DTO — séparation entre entités et API	26
4.18 Réalisation du frontend	26
Dashboard	26
Gestion du matériel	27
Gestion des utilisateurs	28
Gestion des mouvements	29
Campagnes d'inventaire et interfaces par rôle.....	29
Communication avec l'API	30
4.19 Tests et validation	31
4.20 Problèmes rencontrés et solutions apportées.....	32
Accès refusé à Swagger UI	32
Incohérence des valeurs d'énumération en base de données	32
Erreurs liées à l'utilisation de Lombok.....	32
Gestion de la relation entre un article et son lieu d'affectation	33
Gestion des statuts de campagne d'inventaire	33
Problèmes de synchronisation entre le frontend et le backend	33
Difficultés liées à l'affichage des informations de l'utilisateur connecté	34

Erreurs de compilation lors de l'amélioration du PDF	34
Contrôle des informations générées par l'intelligence artificielle	34
Gestion des anciennes données lors de l'évolution du modèle.....	34
4.21 Bilan du projet.....	35
Résultats obtenus	35
Compétences techniques acquises	35
Compétences méthodologiques acquises	35
Limites actuelles du projet	36
Perspectives d'amélioration.....	36
Conclusion du chapitre.....	36
Conclusion générale.....	37
Bibliographie / Webographie	38
Annexes	41
Annexe A — Diagrammes UML complémentaires.....	41
Annexe B — Captures Swagger complémentaires	42
Annexe C — Autres documents techniques	43

Liste des figures

— Organisation de l'INRH.....	2
— Diagramme de cas d'utilisation — vue globale.....	10
— Architecture générale — vue en couches.....	11
— Diagramme de classes — entités et relations du système	14
— Diagramme de séquence — création d'un mouvement	15
— Diagramme de séquence — authentification JWT.....	15
— Statistiques du tableau de bord	20
— Interface Swagger UI.....	21
— Bouton Authorize dans Swagger UI.....	21
— Connexion et récupération du jeton JWT	22
— Test de l'API GET /articles avec Swagger	23
— Génération du rapport assisté par IA	24
— Rapport d'inventaire généré	24
— Aperçu du rapport PDF généré	25
— Dashboard administrateur	26
— Interface de gestion des articles.....	27
— formulaire de création d'Article	27
— Formulaire de création d'utilisateur	28
— Tableau des utilisateurs	28
— Gestion des mouvements	29
— Interface responsable	30
— Interface agent d'inventaire.....	30
— Tests Postman.....	31
-Diagramme activité campagne inventaire	41
-Diagramme séquence générations rapport IA.....	42
-Structure des tables principales	42
-Architecture en couches de l'application Spring Boot	43
-Extrait du prompt ou du contexte envoyé à Ollama	43
-Extrait de la configuration SecurityConfig.....	44
-Architecture de l'application Next.js	44

Liste des tableaux

— <i>Déroulement chronologique du stage</i>	3
— <i>Cas valides et invalides pour l'affectation d'un responsable</i>	8
— <i>Synthèse des exigences fonctionnelles par acteur</i>	9
— <i>Attributs de l'entité Article</i>	13
— <i>Types de mouvements</i>	18
— <i>États du matériel et mouvements autorisés</i>	19
— <i>Exemples de tests réalisés avec Postman</i>	31

Liste des acronymes

Acronyme	Signification
INRH	Institut National de Recherche Halieutique
CSZIAM	Centre Spécialisé en Zootechnie et Ingénierie Aquacoles Marines
API	Application Programming Interface
REST	Representational State Transfer
DTO	Data Transfer Object
JPA	Java Persistence API
JWT	JSON Web Token
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
UML	Unified Modeling Language
UI	User Interface
CRUD	Create Read Update Delete
SQL	Structured Query Language
IA	Intelligence Artificielle
PDF	Portable Document Format
LLM	Large Language Model (modèle de langage utilisé localement via Ollama)

Introduction générale

La gestion du patrimoine matériel constitue un enjeu important pour tout organisme scientifique ou technique disposant d'équipements coûteux et spécialisés. Dans un centre de recherche, ce patrimoine comprend aussi bien du matériel de laboratoire que des équipements techniques répartis entre plusieurs structures : laboratoires, services organisationnels, et utilisateurs auxquels le matériel est affecté.

Lorsque cette gestion repose sur des méthodes manuelles ou dispersées, plusieurs difficultés apparaissent rapidement : les informations sont éclatées entre plusieurs supports, la localisation exacte d'un équipement n'est pas toujours connue, les transferts entre structures sont mal suivis, l'historique des mouvements se perd avec le temps, et l'inventaire annuel devient une source d'erreurs plutôt qu'un contrôle fiable. C'est dans ce contexte que s'inscrit le présent stage, réalisé au sein du Centre CSZIAM, une structure spécialisée de l'INRH.

Le stage, d'une durée de deux mois, avait pour objectif de développer une application web permettant de centraliser et de gérer l'ensemble du patrimoine matériel du centre. Au-delà de la simple gestion des données, le projet devait permettre de suivre les mouvements du matériel, de faciliter l'organisation de l'inventaire annuel, de sécuriser l'accès à l'application selon le rôle de chaque utilisateur, et de fournir des statistiques utiles au suivi du patrimoine, notamment à travers un rapport d'inventaire généré automatiquement.

Pour répondre à ce besoin, une solution a été conçue et développée autour d'un backend Java 21 / Spring Boot exposant une API REST sécurisée par authentification JWT, connecté à une base de données MySQL via Spring Data JPA, et d'un frontend développé avec Next.js, React et TypeScript. Le projet intègre également une documentation des API avec Swagger/OpenAPI, une génération de rapport d'inventaire assistée par intelligence artificielle à l'aide d'un modèle local exécuté avec Ollama, ainsi qu'une génération de document PDF avec la bibliothèque OpenPDF.

La démarche suivie tout au long du stage a consisté à analyser d'abord le besoin réel du centre, à concevoir ensuite le système à travers une modélisation UML, puis à développer progressivement le backend et le frontend, avant de documenter et de tester l'ensemble à l'aide de Postman et de Swagger. Cette démarche a été jalonnée de plusieurs difficultés techniques, présentées dans ce rapport avec les solutions qui leur ont été apportées.

Ce rapport est structuré en quatre chapitres, précédés de la présente introduction générale et suivis d'une conclusion générale. Le premier chapitre présente l'organisme d'accueil, le Centre CSZIAM et le contexte général du stage. Le deuxième chapitre détaille la spécification des besoins : problématique, acteurs, fonctionnalités attendues et règles métier. Le troisième chapitre présente la conception du système : architecture générale, modélisation des données et choix technologiques. Le quatrième chapitre, le plus détaillé, présente la réalisation complète du système, les tests effectués, les problèmes rencontrés et les solutions apportées, ainsi que le bilan du projet. Le rapport se termine par une conclusion générale, une bibliographie et des annexes.

Chapitre 1 — Présentation de l'organisme et du contexte

1.1 Introduction

Ce premier chapitre présente le cadre général dans lequel ce stage s'est déroulé : l'organisme d'accueil, le Centre CSZIAM pour lequel le projet a été réalisé, le déroulement chronologique du stage, le sujet confié, la problématique posée, ainsi que les objectifs fixés au départ.

1.2 Présentation de l'organisme d'accueil

L'Institut National de Recherche Halieutique (INRH) est un établissement public à caractère scientifique et technique, doté de la personnalité morale et de l'autonomie financière nécessaires à la réalisation de ses programmes de recherche scientifique. Ses missions couvrent notamment la conduite d'études et de recherches destinées à approfondir les connaissances sur le milieu marin, la surveillance continue de l'état de l'environnement marin et des sources de pollution, l'évaluation des ressources halieutiques et de leur niveau d'exploitation biologique, ainsi que l'évaluation de l'impact biologique et socio-économique des mesures d'aménagement du littoral.

L'INRH assure également la surveillance de la salubrité des produits de la pêche et de l'aquaculture, évalue les potentialités aquacoles du littoral national, et mène des recherches en aquaculture ainsi qu'en technologie des pêches. Il assure enfin la diffusion de la connaissance scientifique et participe à la coopération scientifique nationale, régionale et internationale.

L'INRH dispose d'une organisation fortement décentralisée : ses laboratoires de recherche sont répartis entre six centres régionaux multidisciplinaires et trois centres spécialisés, implantés le long du littoral marocain à Nador, Tanger, Casablanca, Agadir, Laâyoune et Dakhla. Le siège de l'INRH regroupe la direction, le département support et les départements scientifiques, dédiés à la planification stratégique et au pilotage global des activités scientifiques au niveau national.

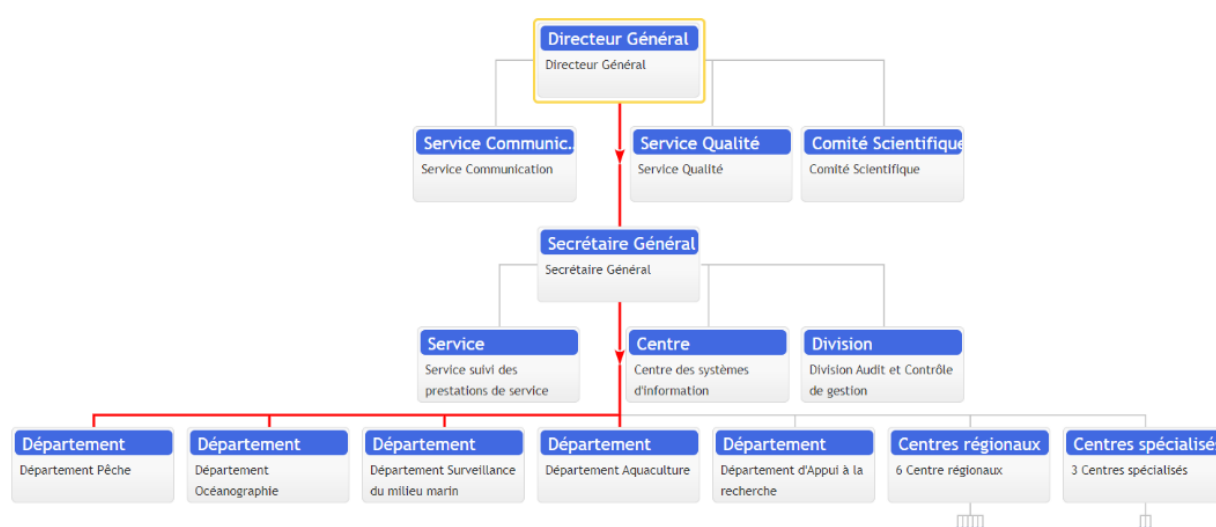


Figure 1 — Organisation de l'INRH

1.3 Présentation du Centre CSZIAM

Le Centre Spécialisé en Zootechnie et Ingénierie Aquacoles Marines (CSZIAM), basé à M'diq et rattaché à l'INRH, constitue l'infrastructure nationale de référence pour le développement de l'aquaculture marocaine. Le centre a pour rôle principal de piloter la recherche scientifique et l'innovation technologique appliquées aux élevages marins, autour de trois axes majeurs : la recherche zootechnique, portant sur la maîtrise des cycles de reproduction, de la nutrition et de la génétique des poissons et des coquillages ; l'ingénierie aquacole, portant sur la conception, le test et l'optimisation des structures d'élevage adaptées aux conditions côtières marocaines ; et l'appui technique et la vulgarisation, à travers le transfert de technologies et de savoir-faire vers les investisseurs privés et les coopératives locales.

Pour assurer ces missions scientifiques et opérationnelles, le CSZIAM s'appuie sur un patrimoine matériel hautement spécialisé et coûteux. La gestion rigoureuse de ce patrimoine est indispensable pour plusieurs raisons. D'une part, la continuité de la recherche et la sécurité du vivant en dépendent directement : le centre abrite des laboratoires de pointe, des éclosiers et des bassins d'élevage, et une panne sur les systèmes de filtration, d'aération ou de thermorégulation peut détruire instantanément des années de recherche ainsi que des stocks de géniteurs précieux. D'autre part, l'optimisation des coûts opérationnels est en jeu : les équipements d'ingénierie marine et le matériel de plongée scientifique subissent une forte corrosion en milieu marin, et un suivi patrimonial strict, incluant l'inventaire et la maintenance préventive, prolonge la durée de vie des actifs et réduit les dépenses de renouvellement. Enfin, la fiabilité des données scientifiques dépend du maintien en parfait état de calibrage des instruments de mesure et d'analyse, condition nécessaire à la précision et à la crédibilité internationale des résultats de recherche de l'INRH.

1.4 Contexte et déroulement du stage

Le stage a duré deux mois, du 13 juillet au 13 septembre 2026, soit environ huit semaines. Le sujet du projet a été officiellement attribué autour du 20 juillet, bien que certaines activités préparatoires aient débuté avant cette date, notamment la prise en main de l'environnement de travail.

Le tableau ci-dessous présente le déroulement chronologique approximatif du stage. Les dates précises de chaque tâche n'étant pas toutes documentées avec certitude, les travaux sont regroupés par période présentés semaine par semaine de façon artificiellement précise.

Tableau 1 — Déroulement chronologique du stage

Période	Travaux réalisés
Semaine 1	Prise de connaissance de l'environnement professionnel et du fonctionnement général du centre ; découverte du contexte de travail et du domaine de l'inventaire ; observation des besoins liés à la gestion du patrimoine ; recherche et apprentissage des technologies et concepts nécessaires au projet afin de préparer les étapes de développement.

Période	Travaux réalisés
Semaine 2	Après la définition du sujet, étude du cahier des charges et compréhension des fonctionnalités attendues ; identification des principaux acteurs et de leurs rôles ; conception des premiers diagrammes UML ; mise en place du projet Spring Boot et début du développement du backend ; création des premières entités et fonctionnalités de gestion.
Semaine 3	Poursuite et finalisation progressive du backend ; intégration de la base de données MySQL et mise en place des relations entre les différentes entités ; développement des opérations CRUD et des premières règles métier ; début de la mise en place de l'authentification avec JWT et Spring Security ; réalisation de tests des API au fur et à mesure du développement.
Semaine 4	Finalisation de l'authentification et de la sécurisation des API avec JWT et Spring Security ; poursuite des tests et corrections du backend ; configuration de Swagger/OpenAPI pour faciliter la documentation et les tests des endpoints ; recherche et étude de références d'interfaces afin de préparer la conception du frontend ; définition de la structure générale des différentes interfaces.
Semaine 5	Début du développement du frontend avec Next.js ; création de l'interface de connexion et mise en place de la structure générale de l'application ; développement progressif du tableau de bord administrateur et des principales interfaces ; connexion progressive entre le frontend et les API du backend.
Semaine 6	Poursuite du développement du frontend et création des différentes interfaces destinées aux administrateurs, responsables et agents d'inventaire ; intégration des fonctionnalités de gestion des articles, catégories, services, laboratoires, campagnes et mouvements ; ajustements du backend en fonction des besoins rencontrés lors de l'intégration avec le frontend ; tests et corrections effectués tout au long du développement.
Semaine 7	Finalisation et amélioration des principales interfaces frontend ; vérification du fonctionnement global des différentes fonctionnalités et des rôles utilisateurs ; nettoyage et ajustement des données de test afin d'obtenir des données cohérentes et représentatives du contexte du projet ; corrections de différentes anomalies rencontrées lors des tests et de l'intégration.
Semaine 8	Proposition et développement d'une fonctionnalité complémentaire de génération de rapports assistée par intelligence artificielle à l'aide d'Ollama ; préparation des données statistiques nécessaires à la génération du rapport ; mise en place de la génération et de l'exportation des rapports au format PDF avec OpenPDF ; amélioration de la présentation des documents générés ; réalisation de tests complémentaires et finalisation du projet et du rapport de stage.

Cette chronologie reste une reconstitution approximative de la démarche suivie et non un planning daté avec précision ; elle vise avant tout à montrer la progression logique du projet, de l'analyse initiale jusqu'à la finalisation des dernières fonctionnalités.

1.5 Sujet du stage

Le sujet du stage porte sur le développement d'une application web de gestion d'inventaire. Le but principal est de permettre au Centre CSZIAM de centraliser et de gérer tout son patrimoine matériel à travers une seule application, plutôt que de dépendre de méthodes de suivi dispersées. L'application couvre la gestion des articles, des catégories, des laboratoires et des services organisationnels, la gestion des comptes utilisateurs selon plusieurs rôles, le suivi des mouvements du matériel, la gestion des campagnes d'inventaire annuel, ainsi que la génération de statistiques et de rapports sur l'état du patrimoine.

Le projet comprend également deux fonctionnalités plus avancées : une génération de rapport d'inventaire assistée par intelligence artificielle, qui produit un texte de synthèse à partir des statistiques réellement calculées par l'application, et une génération de document PDF permettant d'obtenir un rapport présentable et structuré, prêt à être partagé ou archivé.

1.6 Problématique

Quand un centre gère son matériel de façon manuelle ou avec des outils dispersés, plusieurs problèmes reviennent souvent : les informations sont éclatées entre plusieurs supports, on ne sait pas toujours où se trouve exactement un équipement, on a du mal à savoir à quelle personne il est affecté, les transferts sont mal suivis, l'historique des mouvements se perd avec le temps, et l'inventaire annuel devient une source d'erreurs plutôt qu'un contrôle fiable.

C'est ce constat qui a mené à la problématique de ce stage : comment concevoir et développer une application web sécurisée permettant de centraliser, suivre et gérer efficacement le patrimoine matériel du Centre CSZIAM, tout en garantissant la traçabilité des affectations et des mouvements, et en facilitant la production de rapports fiables sur l'état du patrimoine ?

1.7 Objectifs du projet

Pour répondre à cette problématique, le projet vise plusieurs objectifs. Le premier est de centraliser toutes les informations relatives au matériel du centre dans une base de données unique, accessible depuis une seule application. Il faut aussi permettre la gestion des catégories, des laboratoires et des services organisationnels, qui servent à situer le matériel dans son contexte d'utilisation.

Un autre objectif important est la gestion des comptes utilisateurs selon plusieurs rôles, chacun avec un accès adapté à ses besoins et protégé par une authentification sécurisée. L'application doit également assurer une traçabilité complète des mouvements du matériel, faciliter la réalisation de l'inventaire annuel à travers des campagnes dédiées, et appliquer des règles de validation côté serveur pour garantir que les données restent cohérentes. Enfin, elle doit permettre de produire des statistiques utiles au suivi

du patrimoine, et de générer un rapport d'inventaire structuré, aussi bien à l'écran que sous forme de document PDF.

1.8 Conclusion

Ce premier chapitre a permis de situer le cadre général du stage : l'organisme d'accueil, le Centre CSZIAM, le déroulement chronologique du travail, ainsi que le sujet, la problématique et les objectifs du projet. Le chapitre suivant détaille la spécification des besoins qui a guidé la conception de l'application.

Chapitre 2 — Spécification des besoins

2.1 Introduction

Avant le début du développement, une analyse approfondie a été menée pour cerner précisément les besoins du centre. Ce chapitre revient sur cette analyse : les acteurs de l'application, les fonctionnalités attendues, les exigences non fonctionnelles, les règles métier à respecter, ainsi que les cas d'utilisation identifiés.

2.2 Identification des acteurs

Administrateur

L'administrateur, ou gestionnaire du patrimoine, a accès à toutes les fonctionnalités de l'application. C'est lui qui gère les utilisateurs, les articles, les catégories, les laboratoires, les services organisationnels, les mouvements et les campagnes d'inventaire. Il peut également valider l'inventaire et exporter les rapports au format PDF. C'est le rôle disposant du plus grand nombre de responsabilités dans l'application.

Responsable de laboratoire ou de service

Le responsable est rattaché à une seule structure : soit un laboratoire, soit un service organisationnel, mais jamais les deux en même temps. Cette contrainte a été l'une des règles métier les plus importantes à implémenter correctement. Le responsable peut consulter le matériel et les équipements associés à sa structure, et se voir affecter des articles sous sa responsabilité.

Agent d'inventaire

L'agent d'inventaire intervient dans les opérations de terrain liées à l'inventaire. Il peut consulter les articles à contrôler, effectuer le pointage d'un article, c'est-à-dire vérifier sa présence physique lors d'une campagne d'inventaire, et signaler un article manquant lorsque celui-ci n'est pas retrouvé lors du contrôle.

2.3 Besoins fonctionnels

L'application doit d'abord permettre de centraliser toutes les informations relatives aux articles et au matériel du centre. Autour de cette base, plusieurs fonctionnalités viennent compléter le système : la gestion des catégories de matériel, la gestion des laboratoires et des services organisationnels, ainsi que la gestion des comptes utilisateurs avec leurs différents rôles.

Au-delà de la simple gestion des données, l'application doit permettre de créer, consulter et suivre les mouvements du matériel dans le temps, de faciliter l'organisation de l'inventaire annuel à travers des campagnes, d'effectuer le pointage des articles lors des contrôles, et de garder une trace fiable des affectations et des déplacements.

Un tableau de bord doit donner une vue d'ensemble utile pour le suivi général du patrimoine, avec des statistiques portant sur les articles, leur état, leur valeur, les mouvements, les inventaires et les campagnes.

Enfin, l'application doit permettre de générer un rapport d'inventaire comprenant un texte de synthèse produit avec l'assistance d'une intelligence artificielle locale, ainsi qu'un export du rapport au format PDF, structuré et présentable. La page dédiée à l'inventaire permet également d'exporter les données d'inventaire au format Excel afin de faciliter leur exploitation et leur archivage.

2.4 Besoins non fonctionnels

Sur le plan de la sécurité, l'accès à l'application est protégé par une authentification basée sur JWT (JSON Web Token) : chaque utilisateur doit s'authentifier via un point d'entrée dédié pour obtenir un jeton, qui est ensuite requis pour accéder aux ressources protégées de l'API. Les mots de passe des utilisateurs ne sont jamais stockés en clair : ils sont chiffrés grâce à un PasswordEncoder. Le mot de passe d'un utilisateur n'est par ailleurs pas remplacé lors d'une modification si aucun nouveau mot de passe n'est saisi, pour éviter d'écraser accidentellement un mot de passe valide.

Un autre besoin non fonctionnel essentiel concerne la fiabilité des règles métier : elles doivent être vérifiées côté backend, et pas seulement côté interface, pour qu'un utilisateur ne puisse pas les contourner en envoyant directement une requête à l'API. L'API devait également être documentée avec Swagger/OpenAPI et testable facilement avec Postman, pour faciliter à la fois le développement et la validation du travail.

Concernant la génération de rapport assistée par intelligence artificielle, une exigence importante est la limitation des hallucinations : le contenu généré doit rester strictement basé sur les statistiques réellement calculées par l'application, sans introduire d'informations non vérifiées. L'intelligence artificielle ne doit pas prendre de décision à la place de l'utilisateur ; elle sert uniquement à formuler, sous forme de texte, les statistiques déjà disponibles.

2.5 Règles métier

Règle relative aux responsables

Cette règle a été l'une des plus importantes à gérer côté backend, car elle conditionne la validité même d'un compte responsable.

Tableau 2 — Cas valides et invalides pour l'affectation d'un responsable

Cas	Validité
Responsable + laboratoire (uniquement)	Valide
Responsable + service (uniquement)	Valide
Responsable + laboratoire + service	Invalide
Responsable sans laboratoire ni service	Invalide

Règle relative aux états du matériel

Un matériel dont l'état est incompatible avec une opération donnée ne peut pas subir certains mouvements. Par exemple, un matériel en panne ne peut être ni affecté, ni prêté, ni transféré. Les états maintenance et en réparation bloquent eux aussi les mouvements incompatibles avec l'état du matériel. Quand un matériel est hors service, seule une sortie définitive ou une mise au rebut reste possible.

Toutes ces règles sont vérifiées côté backend, ce qui garantit qu'elles s'appliquent même si quelqu'un essaie de les contourner en envoyant directement une requête HTTP à l'API sans passer par l'interface. Le frontend, de son côté, affiche des messages clairs pour expliquer à l'utilisateur pourquoi une action lui est refusée.

2.6 Synthèse des exigences par acteur

Le tableau ci-dessous résume, pour chaque acteur identifié, les principales exigences fonctionnelles qui lui correspondent. Cette synthèse permet de vérifier que chaque besoin exprimé est bien rattaché à au moins un acteur, et qu'aucun acteur ne se voit attribuer une fonctionnalité qui ne le concerne pas.

Tableau 3 — Synthèse des exigences fonctionnelles par acteur

Acteur	Exigences principales
Administrateur	Gestion des utilisateurs, des articles, des catégories, des laboratoires et des services ; création des campagnes d'inventaire ; validation et exporter les données d'inventaire au format Excel ; consultation des statistiques ; export du rapport en PDF.
Responsable de laboratoire ou de service	Consultation du matériel et des équipements rattachés à sa structure ; réception d'articles affectés par l'administrateur.
Agent d'inventaire	Consultation des articles à contrôler ; pointage des articles lors d'une campagne ; signalement des articles manquants.

2.7 Cas d'utilisation

Le diagramme ci-dessous présente les principales interactions entre les trois acteurs de l'application (Administrateur, Responsable de laboratoire ou de service, Agent d'inventaire) et le système. Il a été réalisé lors de la phase d'analyse, avant le début du développement.

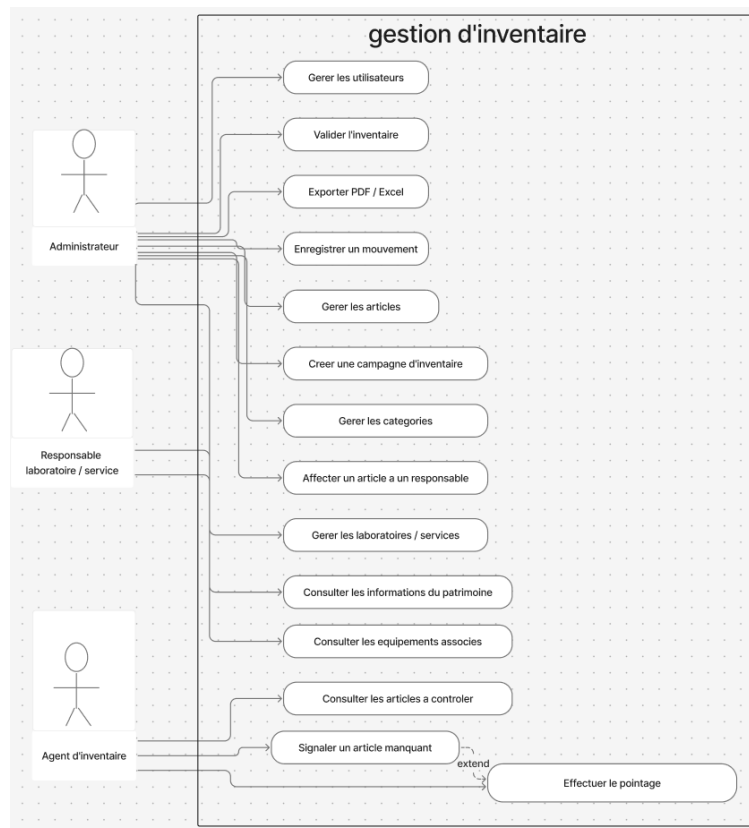


Figure 2 — Diagramme de cas d'utilisation — vue globale

L'administrateur dispose des cas d'utilisation les plus étendus : gérer les utilisateurs, gérer les articles, gérer les catégories, gérer les laboratoires et les services, créer une campagne d'inventaire, enregistrer un mouvement, valider l'inventaire, et exporter les données. Le responsable de laboratoire ou de service peut, quant à lui, affecter un article, consulter les informations du patrimoine et consulter les équipements associés à sa structure. L'agent d'inventaire dispose de cas d'utilisation plus ciblés : consulter les articles à contrôler, signaler un article manquant, et effectuer le pointage, ce dernier cas étant relié au signalement par une relation d'extension : le pointage peut, selon le résultat du contrôle, déclencher un signalement d'article manquant.

2.8 Conclusion

Cette phase d'analyse a permis de poser clairement les bases du projet : les acteurs, les besoins fonctionnels et non fonctionnels, et surtout les règles métier à respecter impérativement. C'est sur cette base qu'a pu commencer la conception détaillée, présentée dans le chapitre suivant.

Chapitre 3 — Conception du système

3.1 Introduction

Ce chapitre présente la conception technique de l'application : son architecture générale, la modélisation des données, les diagrammes UML réalisés, et les choix technologiques retenus avant le début du développement.

3.2 Architecture générale

L'application suit une architecture client-serveur organisée en couches. Le client, développé avec Next.js, communique avec une API REST exposée par le backend Spring Boot. Les contrôleurs reçoivent les requêtes et les transmettent aux services, qui contiennent la logique métier. Les services s'appuient ensuite sur les repositories, via Spring Data JPA, pour accéder à la base de données MySQL.

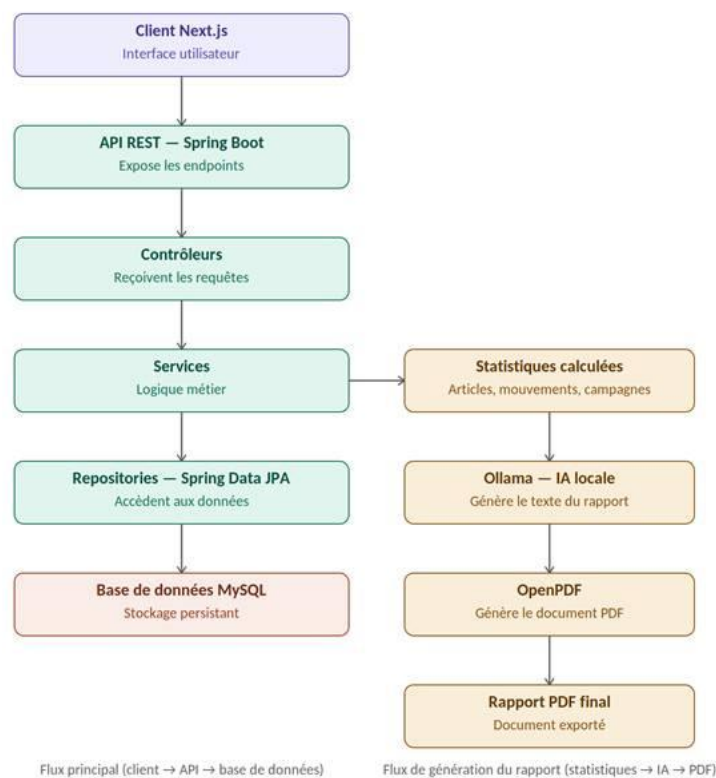


Figure 3 — Architecture générale — vue en couches

Le flux général peut être résumé ainsi : client Next.js, puis API REST Spring Boot, puis services, puis repositories via Spring Data JPA, puis base de données MySQL. Chaque couche a un rôle précis, ce qui rend le code plus facile à comprendre et à faire évoluer par la suite.

À cette architecture principale s'ajoute un second flux, déclenché depuis la couche Services, dédié à la génération du rapport d'inventaire : les statistiques calculées à partir des données de l'application sont transmises à un modèle d'intelligence artificielle exécuté localement avec Ollama, qui génère le texte

de synthèse du rapport ; ce contenu est ensuite mis en forme et exporté sous forme de document PDF grâce à la bibliothèque OpenPDF.

3.3 Architecture backend

Le backend est organisé en plusieurs paquets : config, controllers, entities, enums, repositories, services et DTO. Cette séparation permet de bien distinguer la configuration générale, la gestion des requêtes HTTP, le modèle de persistance, la logique métier et l'accès aux données. Cette organisation n'est pas toujours évidente à saisir au début du projet ; elle prend tout son sens au fil du développement, lorsque chaque couche montre concrètement son utilité.

3.4 Architecture frontend

Le frontend est développé avec Next.js et React, en TypeScript, avec Tailwind CSS pour le style visuel et Lucide React pour les icônes. L'interface s'organise autour d'une barre de navigation latérale (sidebar), avec des pages différentes selon la fonctionnalité et selon le rôle de la personne connectée.

3.5 Pourquoi ces choix technologiques

Spring Boot a été retenu pour le backend en raison de sa maturité et de sa large utilisation en entreprise, qui simplifient considérablement la mise en place d'une API REST : gestion de la configuration, intégration avec Spring Data JPA pour la base de données, validation des données, et sécurité avec Spring Security. Plutôt que de multiplier le code technique répétitif, Spring Boot permet de se concentrer sur la logique métier du projet.

Pour la sécurisation de l'accès à l'application, l'authentification par JWT a été privilégiée car elle est particulièrement adaptée à une architecture client-serveur découplée : le jeton est transmis à chaque requête, ce qui évite de maintenir une session côté serveur et facilite la communication entre le frontend et le backend.

Pour la base de données, MySQL a été retenu. Il s'agit d'un système relationnel qui correspond bien à la nature des données du projet : les articles, les catégories, les laboratoires, les services et les mouvements sont des entités fortement liées entre elles par des relations claires, ce qu'un modèle relationnel prend naturellement en charge.

Le choix d'une API REST s'explique par sa simplicité et par la séparation claire qu'elle établit entre le frontend et le backend : le frontend n'a besoin de connaître que les routes et les formats de données échangés, sans dépendre de l'implémentation interne du serveur. Cette séparation facilite également les tests, puisque l'API peut être interrogée directement avec Postman ou Swagger, indépendamment de l'interface.

Pour le frontend, Next.js a été retenu car il s'appuie sur React, avec une structure de projet claire et de bonnes performances. Associé à TypeScript, il permet de détecter de nombreuses erreurs dès l'écriture du code plutôt qu'au moment de l'exécution, ce qui contribue à limiter certains bugs liés à des données mal typées entre le frontend et le backend.

Pour la génération de rapport assistée par intelligence artificielle, un modèle local exécuté avec Ollama a été retenu plutôt qu'un service distant. Cette solution permet de générer le texte du rapport directement à partir des statistiques calculées par l'application, sans dépendre d'un service externe. Pour la génération du document PDF, la bibliothèque OpenPDF a été utilisée ; elle est adaptée à un projet Java et permet de contrôler précisément la mise en page du document produit.

Enfin, l'architecture en couches et l'usage des DTO permettent de séparer clairement les responsabilités du système et d'éviter d'exposer directement les entités de la base de données à travers l'API. Et si les règles métier sont systématiquement vérifiées côté backend plutôt que côté frontend, c'est parce que le frontend peut toujours être contourné : n'importe qui peut envoyer une requête directement à l'API sans passer par l'interface, donc la seule vérification vraiment fiable est celle faite côté serveur.

3.6 Modélisation des données

Les entités principales du système sont les suivantes : Utilisateur (avec les rôles Administrateur, Responsable et AgentInventaire), Article, Catégorie, Laboratoire, Service organisationnel, Mouvement, CampagneInventaire, et Inventaire. Chacune de ces entités correspond à un concept clair du métier du centre, ce qui a facilité la conception du modèle de données.

Entité Article

Tableau 4 — Attributs de l'entité Article

Attribut	Description
id	Identifiant unique de l'article
désignation	Nom ou description du matériel
numéro d'inventaire	Référence d'inventaire du matériel
état	État courant (par exemple en panne, maintenance, hors service)
date d'acquisition	Date d'acquisition du matériel
valeur	Valeur du matériel
localisation	Localisation courante du matériel
catégorie / laboratoire / service	Structures associées à l'article

Entité Mouvement

L'entité Mouvement est sans doute la plus centrale du système après l'article lui-même, puisqu'elle est celle qui garantit la traçabilité. Elle conserve la date du mouvement, son type, le motif, l'ancienne et la nouvelle localisation, ainsi que l'article et l'utilisateur concernés.

Entités CampagneInventaire et Inventaire

La campagne d'inventaire regroupe les opérations d'inventaire menées sur une période donnée : elle comporte une année, une date de début, une date de fin, ainsi qu'un statut (avenir,encours,termine, expirée). Pour chaque article contrôlé lors d'une campagne, une fiche d'inventaire conserve la date de vérification, un statut de présence indiquant si l'article a bien été retrouvé, ainsi qu'un commentaire

éventuel. C'est cette entité qui permet à l'agent d'inventaire d'effectuer le pointage d'un article et de signaler, le cas échéant, son absence.

Relations entre les entités

Plusieurs relations structurent le modèle de données. Un utilisateur de rôle Administrateur gère l'ensemble des articles, catégories, laboratoires et services. Un responsable est associé à zéro ou un laboratoire, ou à zéro ou un service, jamais aux deux à la fois, conformément à la règle métier présentée au chapitre précédent, et peut se voir affecter plusieurs articles relevant de sa structure. Un article appartient à une catégorie, et peut être rattaché à un laboratoire ou à un service selon sa localisation. Un article peut faire l'objet de plusieurs mouvements au cours du temps, chaque mouvement étant rattaché à un seul article et, le cas échéant, à un seul utilisateur. Enfin, une campagne d'inventaire regroupe plusieurs fiches d'inventaire, chacune étant rattachée à un seul article contrôlé.

3.7 Diagramme de classes

Le diagramme de classes ci-dessous présente les entités du système, leurs attributs, ainsi que les relations et les cardinalités qui les relient. Il met notamment en évidence l'héritage entre l'entité Utilisateur et ses trois spécialisations (Administrateur, Responsable, AgentInventaire), chacune disposant de méthodes propres à son rôle.

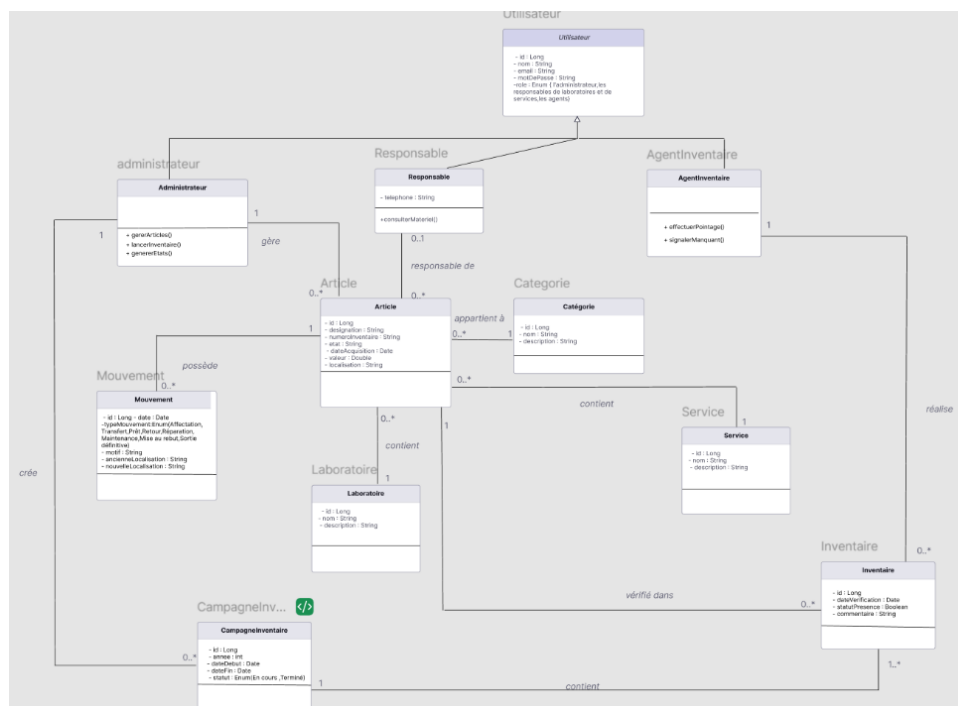


Figure 4 — Diagramme de classes — entités et relations du système

3.8 Diagrammes de séquence

Des diagrammes de séquence permettraient d'illustrer plus en détail le fonctionnement interne de certains scénarios clés de l'application, notamment la création d'un mouvement de matériel et le processus d'authentification d'un utilisateur.

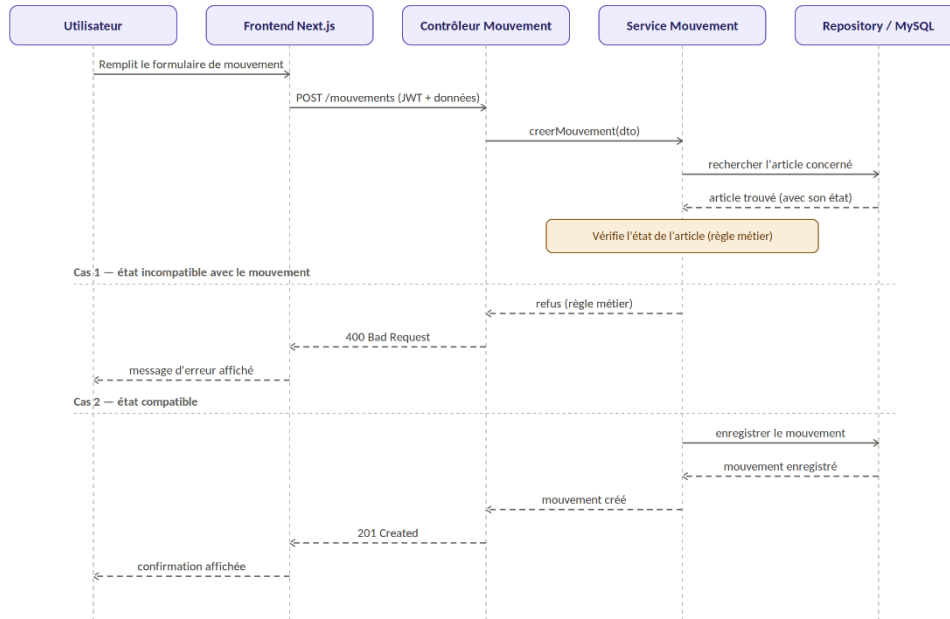


Figure 5— Diagramme de séquence — création d'un mouvement

Ce premier diagramme met en évidence le rôle de la vérification de l'état de l'article : selon que cet état est compatible ou non avec le mouvement demandé, le service renvoie soit un refus (erreur 400), soit confirme l'enregistrement du mouvement (201 Created).

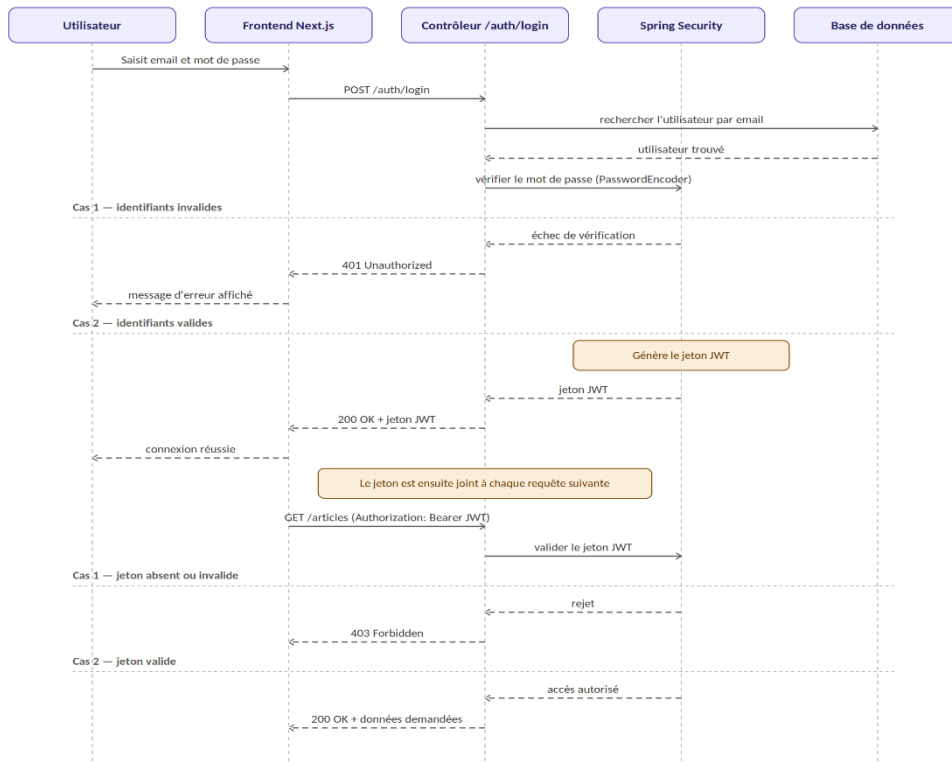


Figure 6— Diagramme de séquence — authentification JWT

Ce second diagramme illustre à la fois l'obtention du jeton JWT via /auth/login et son utilisation ultérieure pour accéder à un endpoint protégé comme /articles : sans jeton valide, l'accès est refusé (403 Forbidden), alors qu'un jeton valide permet d'obtenir la ressource demandée.

3.9 Gestion des rôles

La conception prévoit un contrôle d'accès différent selon le rôle de l'utilisateur connecté. L'administrateur a une vision globale de l'application, le responsable est limité au périmètre de son laboratoire ou de son service, et l'agent d'inventaire dispose d'une interface pensée uniquement pour les opérations d'inventaire : consultation des articles à contrôler, pointage et signalement.

3.10 Conclusion

Cette étape de conception a permis de structurer clairement le développement à venir, aussi bien côté backend que côté frontend. Le chapitre suivant détaille la réalisation concrète du système, les tests effectués, ainsi que le bilan du projet.

Chapitre 4 — Réalisation du système

4.1 Introduction

Ce chapitre présente la réalisation complète du système : l'environnement technique utilisé, le développement du backend et du frontend, la sécurité, la documentation des API, la génération du rapport d'inventaire assisté par intelligence artificielle, la génération du document PDF, les tests réalisés, les problèmes rencontrés et les solutions apportées, ainsi que le bilan du projet.

4.2 Environnement et technologies

Le backend de l'application a été développé en Java 21 à l'aide du framework Spring Boot. La persistance des données repose sur Spring Data JPA, avec une base de données MySQL. L'utilisation de Lombok permet de réduire le code répétitif au niveau des entités, notamment grâce à la génération automatique des getters, setters et constructeurs. La validation des données est assurée par les mécanismes de Bean Validation, afin de contrôler la conformité des informations transmises à l'application.

Concernant la sécurité, Spring Security a été intégré au projet afin de sécuriser les différentes ressources de l'application et de gérer les accès selon les rôles des utilisateurs. L'authentification repose sur JWT (JSON Web Token) : un utilisateur s'authentifie via un point d'entrée dédié et reçoit un jeton, qui doit ensuite être fourni pour accéder aux ressources protégées de l'API. La documentation et le test des API REST ont été réalisés à l'aide de Swagger/OpenAPI, tandis que Postman a été utilisé tout au long du développement pour tester les différents endpoints et vérifier le bon fonctionnement des opérations CRUD ainsi que des règles métier. La génération du rapport d'inventaire s'appuie sur un modèle d'intelligence artificielle exécuté localement avec Ollama, et la génération du document PDF utilise la bibliothèque OpenPDF.

4.3 Organisation du backend

Le code backend est structuré en paquets config, controllers, entities, enums, repositories, services et DTO. Les contrôleurs reçoivent les requêtes HTTP et les transmettent aux services, qui contiennent toute la logique métier et font appel aux repositories pour accéder aux données. Cette organisation, même si elle demande un peu de rigueur au départ, rend le code beaucoup plus facile à faire évoluer par la suite : si une règle métier change, il suffit généralement de modifier le service concerné sans toucher au reste.

4.4 Gestion des utilisateurs

L'administrateur peut créer, consulter, modifier et supprimer des utilisateurs. Les informations gérées comprennent le nom, l'email, le mot de passe, le téléphone, le rôle, ainsi que le service ou le laboratoire d'affectation pour un responsable.

La règle qui garantit qu'un responsable n'est jamais affecté à la fois à un laboratoire et à un service est vérifiée côté backend, au moment de la création ou de la modification du compte. Autre détail important : lors de la modification d'un utilisateur, si aucun nouveau mot de passe n'est saisi, l'ancien mot de passe

reste inchangé plutôt que d'être écrasé par une valeur vide. Et dans tous les cas, les mots de passe sont stockés sous forme chiffrée grâce à un PasswordEncoder, jamais en clair.

4.5 Gestion des articles et du matériel

La gestion des articles permet à l'administrateur de créer, consulter, modifier et supprimer les fiches de matériel : désignation, numéro d'inventaire, état, date d'acquisition, valeur, catégorie, laboratoire et service associés. Cette gestion constitue le socle de l'ensemble de l'application, puisque toutes les autres fonctionnalités — mouvements, inventaires, campagnes, statistiques — s'appuient sur les informations enregistrées ici.

4.6 Gestion des catégories, laboratoires et services organisationnels

En complément de la gestion des articles, l'application permet de gérer les structures qui servent à les organiser : les catégories, qui regroupent les articles par type de matériel, les laboratoires et les services organisationnels, qui représentent les différentes structures du centre auxquelles un article ou un utilisateur peut être rattaché. Chacune de ces structures dispose de ses propres opérations de création, consultation, modification et suppression, accessibles à l'administrateur. Ces structures sont ensuite réutilisées dans plusieurs autres modules : l'affectation d'un responsable, le rattachement d'un article, ou encore le filtrage des statistiques du tableau de bord.

4.7 Validation des données

La validation des données transmises à l'API repose sur les mécanismes de Bean Validation, appliqués directement sur les DTO de requête. Elle permet de vérifier, avant tout traitement métier, que les champs obligatoires sont bien renseignés, que les formats attendus sont respectés (par exemple pour une adresse email), et que les valeurs numériques ou les dates transmises restent cohérentes. Lorsqu'une validation échoue, l'API retourne une erreur explicite plutôt que de laisser une donnée invalide être enregistrée en base de données, ce qui contribue à la fiabilité générale de l'application.

4.8 Gestion des mouvements

L'entité Mouvement conserve la date, le type de mouvement, le motif, l'ancienne et la nouvelle localisation, ainsi que l'article et l'utilisateur liés à ce mouvement.

Tableau 5 — Types de mouvements

Type de mouvement	Usage
Affectation	Attribution d'un article à une personne ou une structure
Transfert	Déplacement d'un article entre deux structures
Prêt	Prêt temporaire d'un article
Retour	Retour d'un article prêté
Réparation	Envoi de l'article en réparation
Maintenance	Passage de l'article en maintenance

Type de mouvement	Usage
Sortie définitive	Sortie définitive du parc matériel

4.9 Traçabilité des mouvements

L'objectif principal de ce module est de garantir qu'on peut toujours retracer le parcours complet d'un article. Pour chaque mouvement, l'historique conserve l'article concerné, la date, le type de mouvement, l'ancienne et la nouvelle localisation, la personne concernée quand elle existe, et le motif du mouvement.

4.10 Règles métier liées aux états du matériel

Tableau 6 — États du matériel et mouvements autorisés

État du matériel	Mouvements autorisés
En panne	Affectation, prêt et transfert refusés
Maintenance	Mouvements incompatibles bloqués
En réparation	Mouvements incompatibles bloqués
Hors service	Seule la sortie définitive est autorisée

Ces règles sont contrôlées côté backend afin qu'un utilisateur ne puisse pas les contourner en envoyant directement une requête à l'API. De son côté, le frontend affiche un message clair expliquant pourquoi une action a été refusée.

4.11 Gestion des campagnes d'inventaire et du pointage

La campagne d'inventaire encadre les opérations de contrôle menées sur une période donnée. Pour chaque article contrôlé, une fiche d'inventaire enregistre la date de vérification, le statut de présence de l'article, et un commentaire éventuel. C'est dans ce cadre que l'agent d'inventaire consulte la liste des articles à contrôler, effectue le pointage de chaque article — c'est-à-dire confirme sa présence physique — et signale un article manquant lorsque celui-ci n'a pas pu être retrouvé. L'administrateur peut ensuite valider l'inventaire une fois la campagne terminée.

4.12 Statistiques et tableau de bord

Le tableau de bord donne une vue d'ensemble du patrimoine du centre à travers des statistiques calculées à partir des données réellement enregistrées dans l'application. Ces statistiques portent notamment sur le nombre d'articles et leur répartition par état, la valeur totale ou par catégorie du patrimoine, le volume et la nature des mouvements enregistrés, l'avancement des inventaires et des campagnes, ainsi que la répartition des utilisateurs, des catégories, des laboratoires et des services organisationnels.

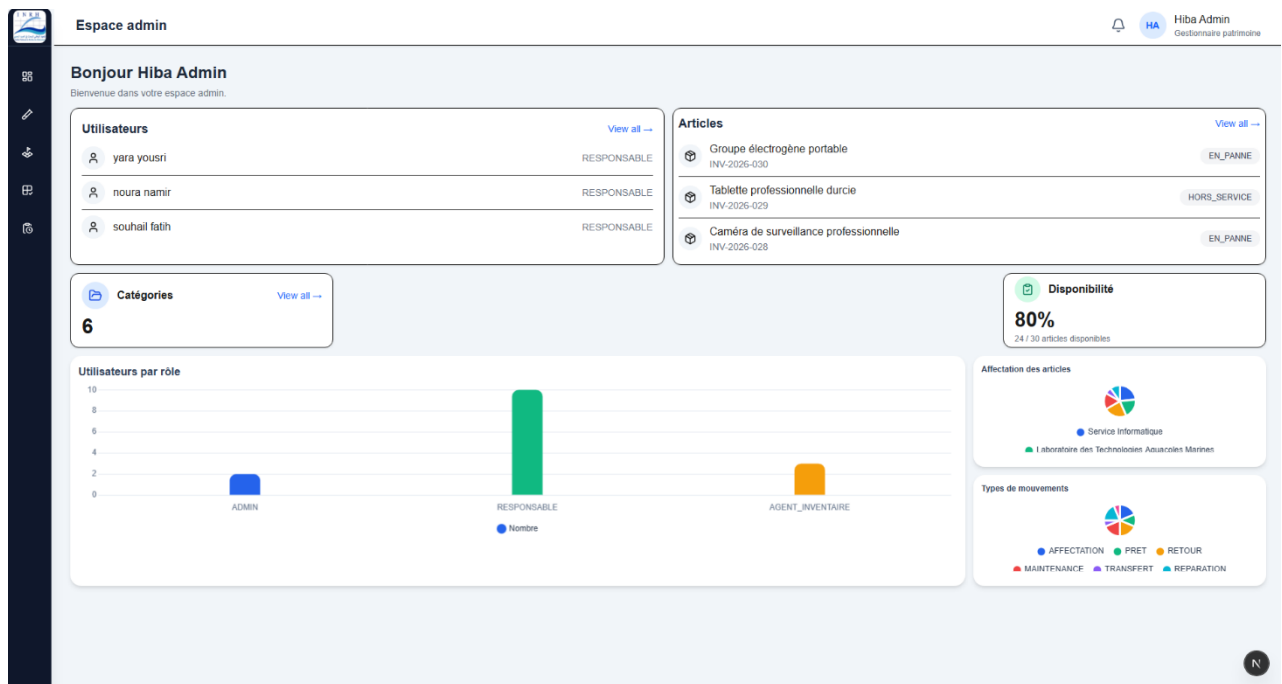


Figure 7 — Statistiques du tableau de bord

4.13 Sécurité et authentification JWT

L'accès à l'API est protégé par Spring Security, configuré pour exiger un jeton JWT valide sur les routes sensibles de l'application. Un utilisateur s'authentifie via le point d'entrée `/auth/login` en fournissant ses identifiants ; en retour, un jeton JWT lui est délivré. Ce jeton doit ensuite être transmis avec chaque requête adressée aux endpoints protégés, comme `/articles`, pour que l'accès soit autorisé.

Cette mise en place a nécessité une configuration précise de Spring Security afin de définir quelles routes nécessitent une authentification et lesquelles restent publiques, notamment les routes liées à Swagger, comme détaillé dans la section consacrée aux problèmes rencontrés.

4.14 Documentation et test des API avec Swagger/OpenAPI

Le projet utilise Springdoc OpenAPI pour générer automatiquement la documentation des API REST et l'exposer à travers une interface Swagger UI accessible depuis le navigateur. Les endpoints y sont organisés par catégories, correspondant globalement aux différents contrôleurs de l'application (utilisateurs, articles, mouvements, campagnes, etc.), avec des titres et des descriptions ajoutés pour faciliter la lecture.

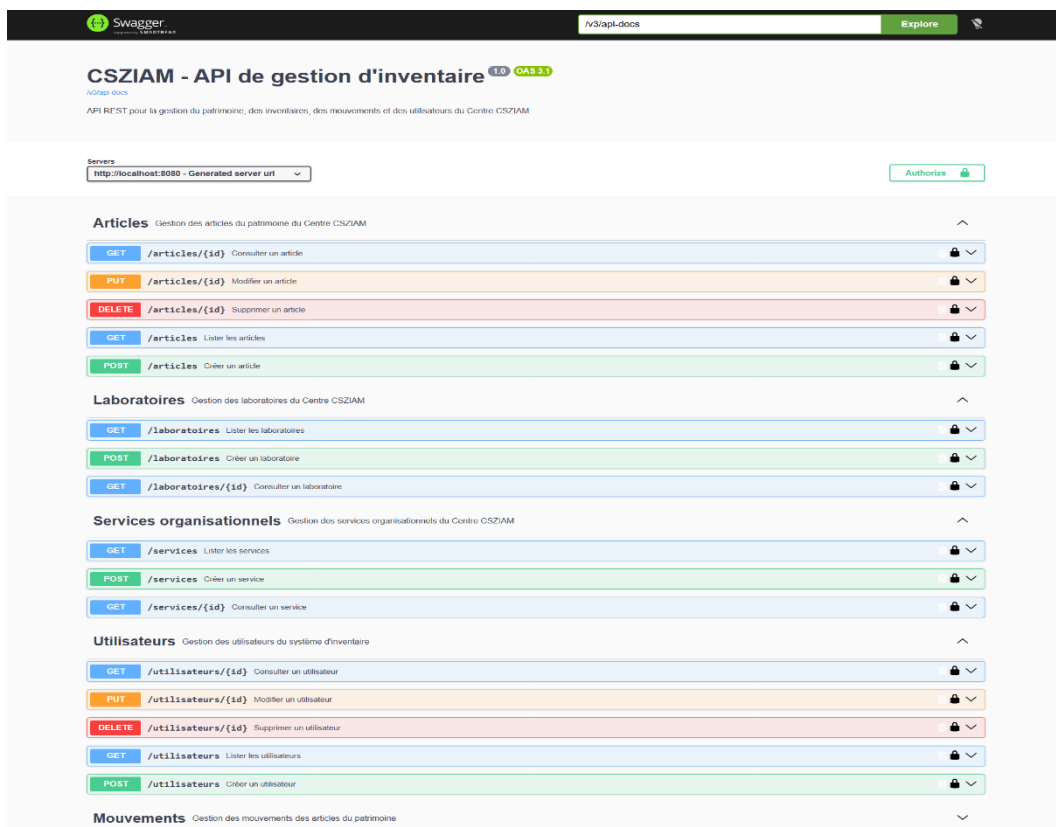


Figure 8 — Interface Swagger UI

Une configuration dédiée, SwaggerApiConfig, définit notamment le schéma de sécurité utilisé par l'API : ce schéma permet à Swagger UI d'afficher un bouton « Authorize » avec une icône de cadenas, à travers lequel un jeton JWT peut être renseigné pour l'ensemble des requêtes envoyées depuis l'interface.

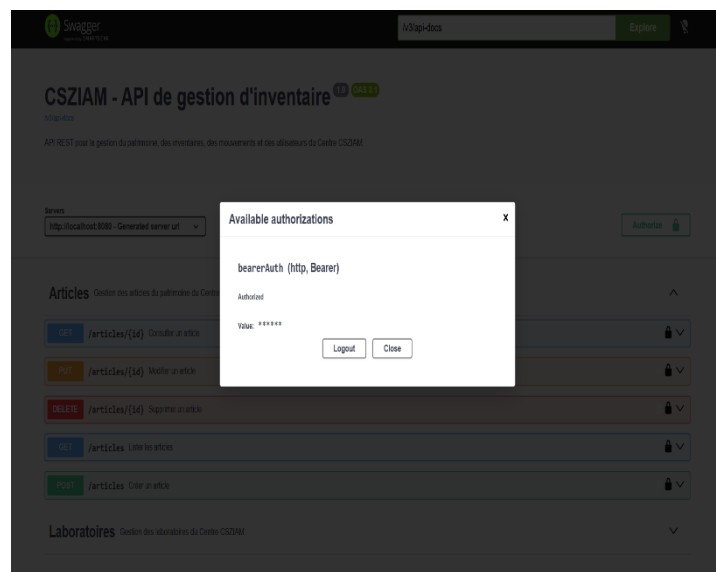
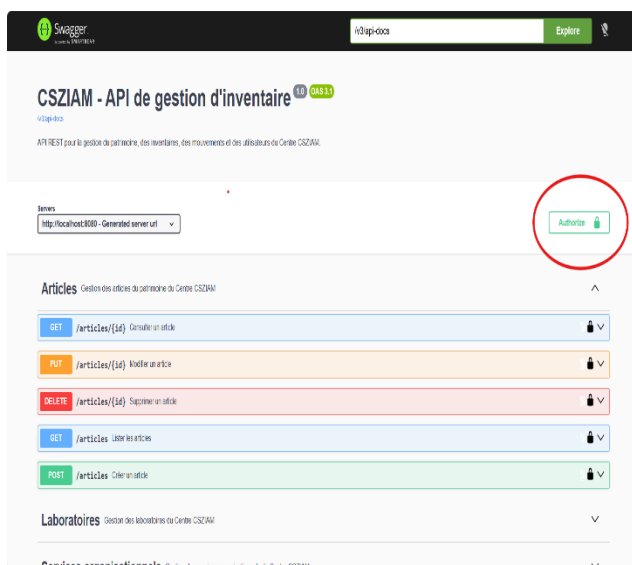


Figure 9 — Bouton Authorize dans Swagger UI

Concrètement, l'utilisation de Swagger pour tester un endpoint protégé se déroule en deux temps. Le point d'entrée /auth/login est d'abord appelé avec des identifiants valides pour récupérer un jeton JWT.

Authentication

Gestion de l'authentification et de l'utilisateur connecté

POST /auth/registerInscrire un utilisateur

POST /auth/loginSe connecter

Authentifie un utilisateur et retourne un token JWT.

Parameters

No parameters

Request bodyrequired

application/json

Edit Value | Schema

{
 "email": "admin@cszlan.ma",
 "motDePasse": "admin123"
}

Execute

Clear

Responses

Curl

```
curl -X 'POST' \  
  'http://localhost:8080/auth/login' \  
  -H 'accept: */*' \  
  -H 'content-type: application/json' \  
  -d '{  
    'email': 'admin@cszlan.ma',  
    'motDePasse': 'admin123'  
  }'
```

Request URL

```
http://localhost:8080/auth/login
```

Server response

CodeDetails

200

Response body

{
 "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpzZW50LmF1dG8iLCJ1eWwiOiJhdXNpdCkiLCJ0eXAiOiJKV1QiLCJhbGciOiJSUzI1NiIsImtkeSI6bnVudDk1MjYyZlN1cS"}
Download

Response headers

cache-control: no-cache,no-store,max-age=0,must-revalidate
connection: keep-alive
content-length: 177
content-type: application/json
date: Sat, 12 Sep 2026 00:31:38 GMT
expires: 0
keep-alive: timeout=60
pragma: no-cache
vary: Origin,Access-Control-Request-Method,Access-Control-Request-Headers
x-content-type-options: nosniff
x-frame-options: DENY
x-xss-protection: 0

Responses

CodeDescriptionLinks

200OKNo links

Media type

"/*"

Controls Accept header

Example Value | Schema

{
 "token": "string"
}

Figure 10 — Connexion et récupération du jeton JWT

Ce jeton est ensuite renseigné dans le bouton Authorize. Sans cette étape, un appel à un endpoint protégé comme GET /articles renvoie une erreur HTTP 403, ce qui a d'ailleurs permis de vérifier que la protection était bien active. Une fois le jeton fourni, le même appel aboutit normalement et retourne la liste des articles attendue.

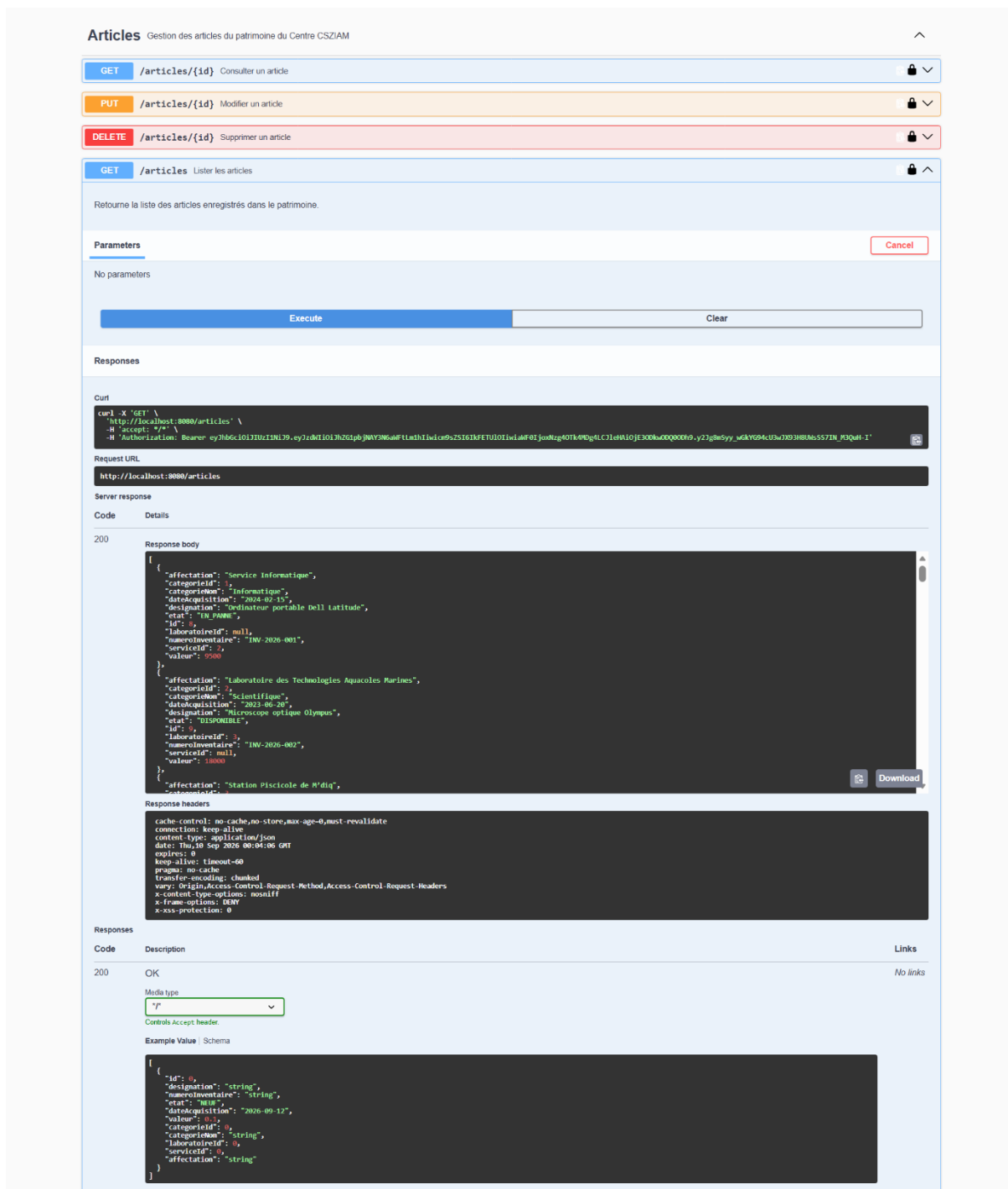


Figure 11 — Test de l'API GET /articles avec Swagger

4.15 Génération du rapport d'inventaire assisté par intelligence artificielle

Le projet comporte une fonctionnalité de génération de rapport d'inventaire, dont la production du texte de synthèse est assistée par intelligence artificielle. Cette fonctionnalité suit une démarche en plusieurs étapes. Les données du système sont d'abord récupérées : articles, états, valeurs, mouvements, inventaires, campagnes, utilisateurs, catégories, laboratoires et services organisationnels. Ces données permettent ensuite de calculer les statistiques nécessaires au rapport.

Une fois les statistiques calculées, elles sont préparées sous une forme exploitable pour la génération du contenu textuel. Le texte du rapport est alors généré avec l'assistance d'un modèle d'intelligence artificielle exécuté localement avec Ollama, à partir de ces statistiques. Une attention particulière est portée au contrôle des informations utilisées par le modèle, afin que le contenu généré reste strictement basé sur les données réelles et n'introduise pas d'informations inventées : l'intelligence artificielle sert uniquement à formuler un texte de synthèse à partir de statistiques déjà calculées par l'application, et ne prend aucune décision à la place de l'utilisateur.

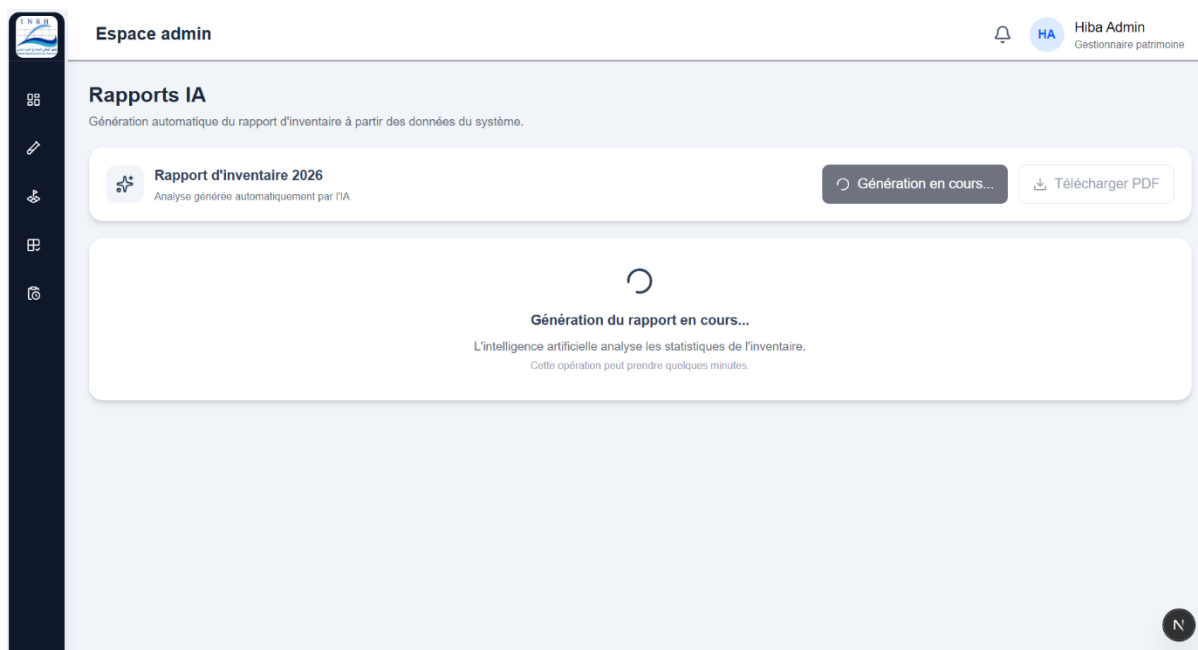


Figure 12 — Génération du rapport assisté par IA

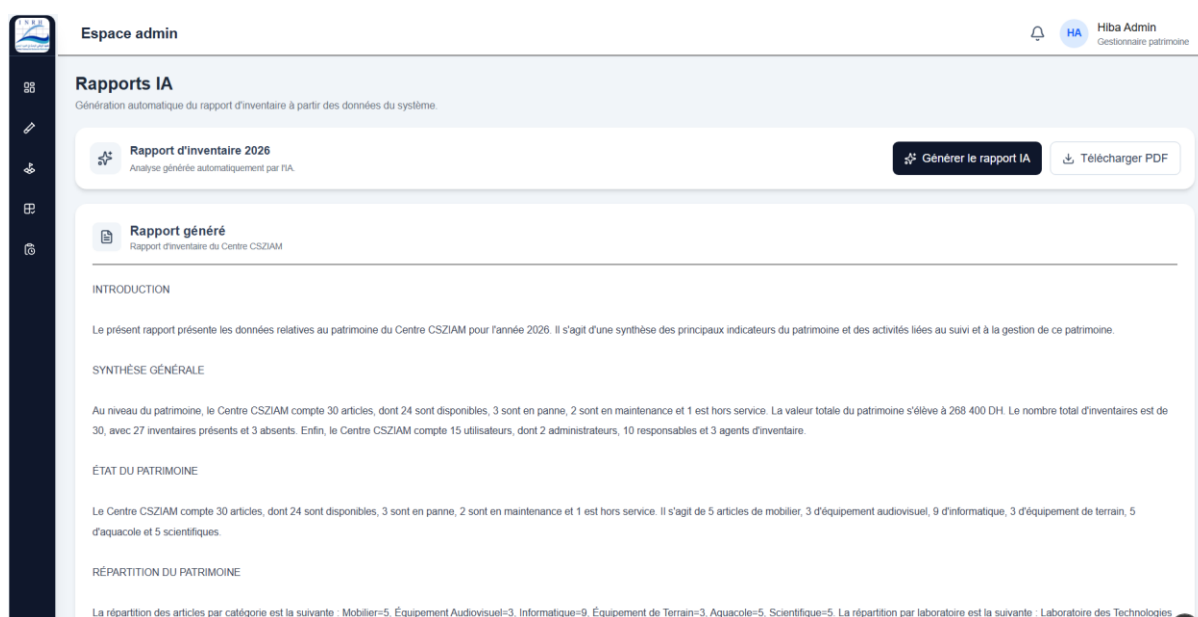


Figure 13 — Rapport d'inventaire généré

Le contenu ainsi obtenu constitue la base du document PDF final, dont la génération est décrite dans la section suivante.

4.16 Génération du document PDF

Une fois le contenu du rapport préparé, un document PDF est généré à l'aide de la bibliothèque OpenPDF. Ce document comprend une page de garde avec le titre du rapport, l'année et un logo, ainsi qu'une mise en page pensée pour rester lisible et professionnelle : couleurs cohérentes, en-tête et pied de page, numérotation des pages, titres de sections, paragraphes structurés, listes, et séparation visuelle claire entre les différentes parties du rapport.

La mise en forme du document a nécessité plusieurs itérations pour améliorer sa lisibilité et sa présentation générale, notamment au niveau de la gestion des couleurs et de la pagination, comme détaillé dans la section consacrée aux problèmes rencontrés.



Figure 14 — Aperçu du rapport PDF généré

4.17 DTO — séparation entre entités et API

Des DTO (Data Transfer Object) ont été utilisés pour séparer les données exposées par l'API des entités JPA utilisées en interne, avec des DTO de requête pour recevoir des données et des DTO de réponse pour en renvoyer. Cette séparation permet de contrôler précisément ce qui entre et sort de l'API, sans exposer directement la structure interne de la base de données.

Aucun mapper automatique de type MapStruct ou ModelMapper n'a été utilisé : les transformations entre entités et DTO sont réalisées manuellement dans le code, ce qui demande davantage d'écriture mais permet de garder un contrôle total sur les données transformées.

4.18 Réalisation du frontend

Le frontend est développé avec Next.js et React, en TypeScript, avec Tailwind CSS pour le style et les icônes Lucide React. L'interface principale s'organise autour d'une sidebar de navigation, ce qui permet à l'utilisateur de passer facilement d'une section à l'autre de l'application.

Dashboard

Le dashboard donne une vue d'ensemble des informations importantes sur le patrimoine matériel du centre, avec les statistiques présentées en section 4.12. C'est en général la première page que voit l'utilisateur en se connectant.

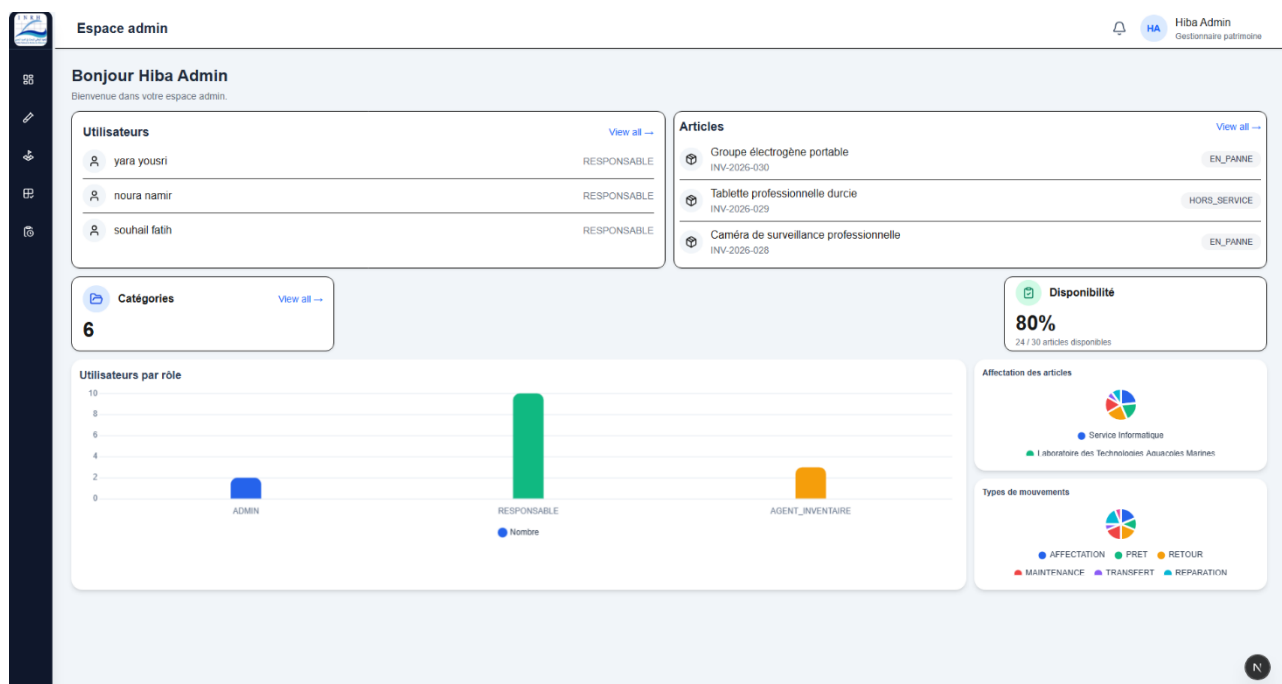


Figure 15 — Dashboard administrateur

Gestion du matériel

La page « Article » permet de consulter et de gérer les articles du centre.

The screenshot shows the 'Gestion des articles' page in the 'Espace admin'. It features a search bar, a table of articles, and a sidebar with navigation icons. The table lists various items with their categories, inventory numbers, states, acquisition dates, values, and assignments.

☐	Désignation	Catégorie	N° d'inventaire	État	Date d'acquisition	Valeur	Affectation	Actions
☐	Ordinateur portable Dell Latitude	Informatique	INV-2026-001	EN_PANNE	2024-02-15	9500	Service Informatique	✎ ✖
☐	Microscope optique Olympus	Scientifique	INV-2026-002	DISPONIBLE	2023-06-20	18000	Laboratoire des Technologies Aquacoles Marines	✎ ✖
☐	Pompe de circulation aquacole	Aquacole	INV-2026-003	DISPONIBLE	2025-01-10	12500	Station Piscicole de M'diq	✎ ✖
☐	Imprimante multifonction HP	Informatique	INV-2026-004	DISPONIBLE	2023-09-05	3200	Service Informatique	✎ ✖
☐	Caméra numérique Sony	Équipement Audiovisuel	INV-2026-005	MAINTENANCE	2024-11-18	7800	Laboratoire des Technologies Aquacoles Marines	✎ ✖
☐	Système de mesure de qualité de l'eau	Scientifique	INV-2026-006	DISPONIBLE	2026-01-22	30000	Laboratoire des Technologies Aquacoles Marines	✎ ✖
☐	Bureau professionnel	Mobilier	INV-2026-007	DISPONIBLE	2025-04-12	4200	Service d'Appui à la Recherche	✎ ✖
☐	Armoire métallique de rangement	Mobilier	INV-2026-008	DISPONIBLE	2024-07-30	3600	Service d'Appui à la Recherche	✎ ✖ N

Figure 16 — Interface de gestion des articles

The screenshot shows the 'Ajouter un article' form, which is used to add a new item to the inventory. The form includes fields for designation, inventory number, state, acquisition date, value, category, and location.

Ajouter un article
Ajouter un nouvel article à l'inventaire

Désignation
Ex: Ordinateur Dell

Numéro d'inventaire
Ex: INV-2026-001

État
Disponible

Date d'acquisition
jj/mm/aaaa

Valeur
0

Catégorie
Sélectionner une catégorie

Emplacement
☒ Service ☐ Laboratoire
Sélectionner un service

Annuler Créer l'article

Figure 17— formulaire de création d'Article

Ce formulaire permet d'enregistrer un nouvel article du centre en renseignant les informations nécessaires telles que la désignation, le numéro d'inventaire, la catégorie, l'état, la date d'acquisition et la valeur de l'article.

Gestion des utilisateurs

Dans le formulaire de création ou de modification d'un utilisateur, quand le rôle sélectionné est Responsable, l'interface propose de choisir soit un laboratoire, soit un service, jamais les deux, pour rester cohérent avec la règle appliquée côté backend. Dans le tableau des utilisateurs, le nom du laboratoire ou du service d'affectation est affiché plutôt que son simple identifiant.

Ajouter un utilisateur

Nom

Email
admin3@csziam.ma

Mot de passe

Téléphone

Rôle
Responsable

Affectation du responsable
Un responsable doit appartenir à un service ou à un laboratoire.

Service
Aucun service

OU

Laboratoire
Aucun laboratoire

Annuler Ajouter

Figure 18 — Formulaire de création d'utilisateur

Espace admin

Gestion des utilisateurs

Rechercher par nom ou email...

+ Ajouter un utilisateur

☐	Nom	Email	Téléphone	Rôle	Actions
☐	Hiba Admin	admin3@csziam.ma	0611111113	ADMIN	
☐	Yassine El Amrani	Amrani@csziam.ma	0620202033	ADMIN	
☐	Salma Bennani	Bennani@csziam.ma	0620202036	RESPONSABLE	
☐	Lina Alaoui	Alaoui@csziam.ma	0610101022	RESPONSABLE	
☐	Sara Tazi	Tazi@csziam.ma	0620202023	RESPONSABLE	
☐	Imane El Idrissi	Idrissi@csziam.ma	0601010101	RESPONSABLE	
☐	Nour Benali	Benali@csziam.ma	0602020202	RESPONSABLE	
☐	Adam Chraïbi	Chraïbi@csziam.ma	0620202022	RESPONSABLE	
☐	Ayoub Mansouri	Mansouri@csziam.ma	0630303037	AGENT_INVENTAIRE	
☐	Mariam Zahra	Zahra@csziam.ma	0604030201	AGENT_INVENTAIRE	
☐	Omar Naciri	Naciri@csziam.ma	0605040302	AGENT_INVENTAIRE	
☐	essia Iamar	lamar@csziam.ma	0609080602	RESPONSABLE	
☐	souhail fatih	fatih@csziam.ma	0620401032	RESPONSABLE	

Figure 19 — Tableau des utilisateurs

Gestion des mouvements

La page « Mouvements » permet de sélectionner un article, de consulter son emplacement actuel, de créer un nouveau mouvement, et de consulter l'historique de tous les mouvements déjà enregistrés pour cet article.

Espace admin

HA

Hiba Admin

Gestionnaire patrimoine

Mouvements

Gestion des déplacements des articles

Articles

Sélectionnez un article pour effectuer un mouvement.

Ordinateur portable Dell Latitude

INV-2026-001

Service Informatique

Microscope optique Olympus

INV-2026-002

Laboratoire des Technologies Aquacoles Marines

Pompe de circulation aquacole

INV-2026-003

Station Piscicole de Mdiq

1-3 sur 30 articles

Nouveau mouvement

Ordinateur portable Dell Latitude

Ancienne localisation

Service Informatique

Action limitée

Cet article est en panne. Il ne peut pas être affecté, prêté ou transféré.

Type de mouvement

Transfert

Date

12/09/2026

Motif

Ex : Réorganisation du laboratoire

Nouvelle localisation

Service

Laboratoire

Sélectionner un service

Annuler

Enregistrer le mouvement

Historique des mouvements

Tous les mouvements enregistrés.

Article	Date	Type	Ancienne localisation	Nouvelle localisation	Responsable(s)	Utilisateur	Motif
Tablette professionnelle durcie ID : 36	08/09/2026	SORTIE_DEFINITIVE	Laboratoire des Technologies Aquacoles Marines	→ Sortie définitive	Adam Chraïbi	—	Matériel réformé
Système de filtration d'eau ID : 33	08/09/2026	MAINTENANCE	Station Conchylicole d'Ansra	→ Maintenance	Nour Benali	—	Maintenance préventive du matériel

Figure 20 — Gestion des mouvements

Campagnes d'inventaire et interfaces par rôle

La page « Campagnes » permet de gérer les campagnes d'inventaire annuel. Une interface spécifique a été développée pour le rôle Responsable, limitée au périmètre de son laboratoire ou de son service. Une interface dédiée à l'agent d'inventaire permet de consulter les articles à contrôler, d'effectuer le pointage et de signaler un article manquant.

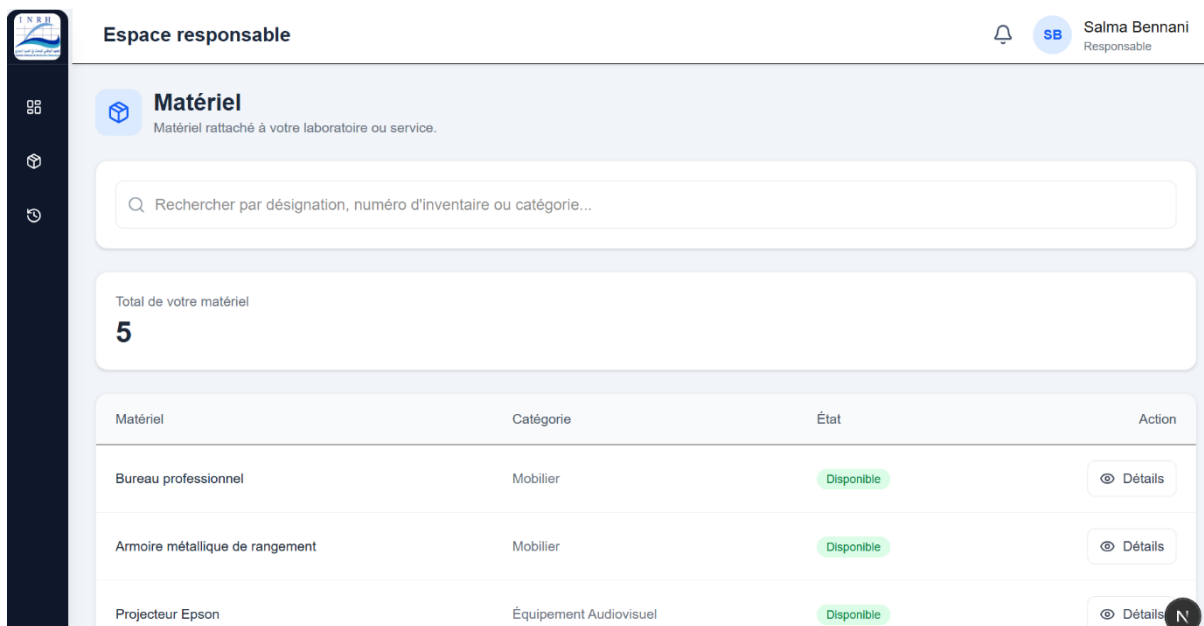


Figure 21 — Interface responsable

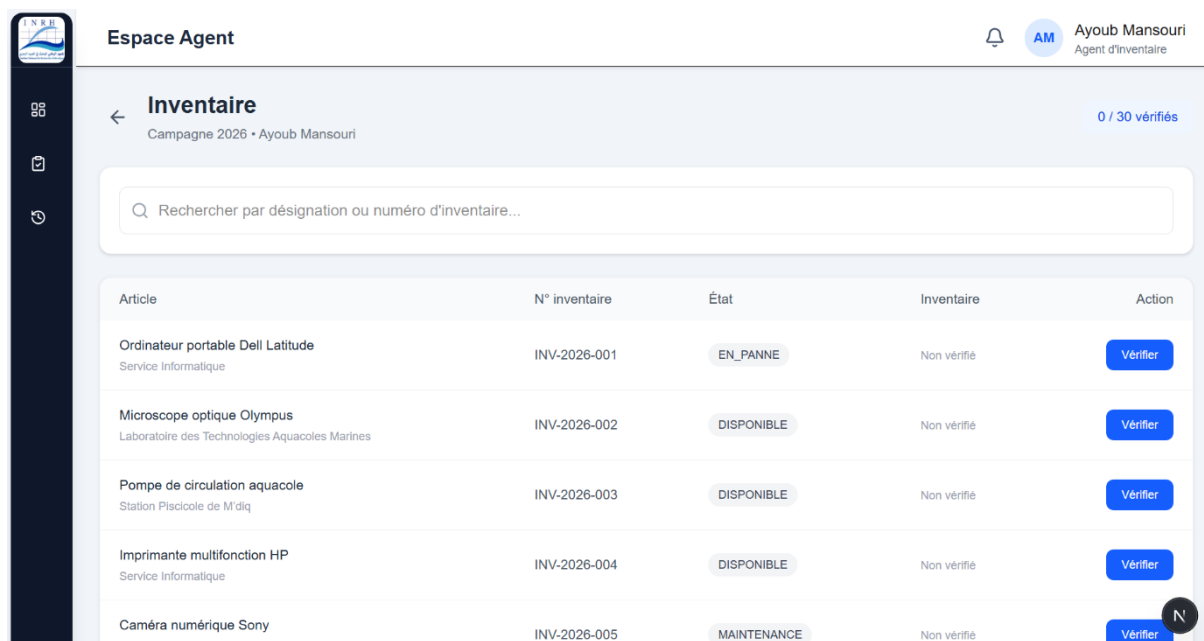


Figure 22 — Interface agent d'inventaire

Communication avec l'API

Le frontend communique avec le backend via des appels REST, en incluant le jeton JWT obtenu lors de la connexion pour accéder aux ressources protégées. Les formulaires envoient les données sous forme de DTO de requête, et l'interface affiche ensuite les réponses ou les messages d'erreur renvoyés par l'API, y compris ceux liés aux règles métier.

4.19 Tests et validation

Les API ont été testées principalement avec Postman, en complément de la documentation Swagger qui permet également d'envoyer des requêtes directement depuis le navigateur. Les tests ont porté sur les opérations de création, de récupération, de modification et de suppression, sur les règles métier, sur les relations entre les entités, ainsi que sur la gestion des mouvements, des affectations et de l'authentification.

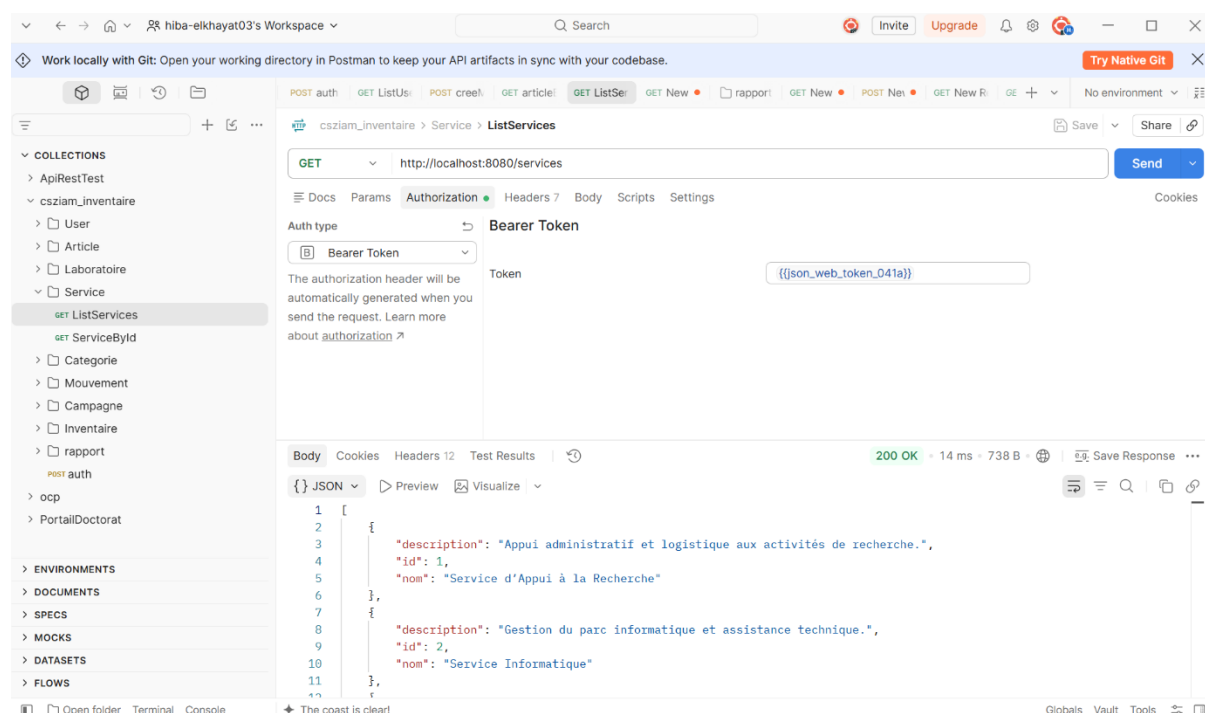


Figure 23 — Tests Postman

Tableau 7 — Exemples de tests réalisés avec Postman

Test	Données	Résultat attendu	Résultat obtenu
Création utilisateur	Données valides	Utilisateur créé	Réussi
Responsable + service	Service valide	Création autorisée	Réussi
Responsable + labo + service	Deux affectations	Refus	Réussi
Responsable sans affectation	Aucun emplacement	Refus	Réussi
Mouvement d'un article en panne	Transfert	Refus	Réussi
Accès à /articles sans jeton JWT	Aucun jeton	Refus (403)	Réussi
Accès à /articles avec jeton JWT valide	Jeton obtenu via /auth/login	Accès autorisé	Réussi

Certaines réponses HTTP obtenues pendant les tests, comme des codes 400, 403 ou 404, ne correspondent pas systématiquement à un échec : plusieurs d'entre elles étaient volontairement renvoyées par le backend, parce qu'elles correspondaient à des règles métier ou à des restrictions de sécurité, comme le refus d'accès sans jeton JWT ou le refus d'un mouvement sur un article dans un état

incompatible. Ces tests ont permis de vérifier que les règles métier et les mécanismes de sécurité définis lors de la conception étaient bien appliqués côté serveur, et pas seulement représentés côté interface.

4.20 Problèmes rencontrés et solutions apportées

Cette section revient sur les principales difficultés techniques réellement rencontrées pendant le développement, en précisant pour chacune le problème, sa cause lorsqu'elle est connue, la solution appliquée et le résultat obtenu.

Accès refusé à Swagger UI

Problème : après l'intégration de Spring Security, l'accès à l'interface Swagger, à l'adresse `/swagger-ui/index.html`, renvoyait une erreur HTTP 403.

Cause : Spring Security bloquait par défaut les routes qui n'étaient pas explicitement autorisées, y compris les routes utilisées par Swagger/OpenAPI pour afficher l'interface et la documentation de l'API.

Solution : les routes correspondant à Swagger et à OpenAPI ont été explicitement autorisées dans la configuration de sécurité (SecurityConfig), afin qu'elles restent accessibles sans authentification, contrairement aux routes métier de l'application.

Résultat : l'interface Swagger UI est redevenue accessible normalement, tout en conservant la protection des routes métier de l'API.

Incohérence des valeurs d'énumération en base de données

Problème : certaines valeurs stockées en base de données pour l'état du matériel, comme d'anciennes valeurs telles que « BON » ou « Neuf », ne correspondaient plus aux valeurs d'énumération réellement utilisées par l'application.

Cause : ces valeurs provenaient de données saisies ou de jeux de test antérieurs à l'évolution des valeurs d'énumération définies dans le code.

Solution : les valeurs incohérentes présentes en base de données ont été identifiées et corrigées afin de correspondre aux valeurs d'énumération actuellement définies dans l'application.

Résultat : la cohérence entre les données stockées et les valeurs attendues par l'application a été rétablie, évitant ainsi les erreurs lors de la récupération et de l'affichage des articles et des mouvements.

Erreurs liées à l'utilisation de Lombok

Problème : lors du développement des entités et des classes du backend, certaines annotations Lombok ne semblaient pas être correctement prises en compte par l'environnement de développement, ce qui provoquait des erreurs de compilation ou l'absence de méthodes générées automatiquement.

Cause : le problème était principalement lié à la configuration de Lombok et à la prise en charge de l'annotation `processing` par l'environnement de développement.

Solution : la configuration de Lombok et de l'annotation `processing` a été vérifiée et corrigée afin de permettre la génération automatique des getters, setters, constructeurs et autres méthodes nécessaires.

Résultat : les classes utilisant Lombok ont pu être compilées et utilisées normalement, ce qui a permis de poursuivre le développement du backend sans devoir écrire manuellement toutes les méthodes générées.

Gestion de la relation entre un article et son lieu d'affectation

Problème : un article pouvait être affecté soit à un laboratoire, soit à un service organisationnel. Il fallait donc éviter qu'un même article soit associé simultanément aux deux types de structures.

Cause : le modèle métier permettait deux possibilités d'affectation différentes, ce qui nécessitait une validation spécifique au niveau du backend.

Solution : une règle de validation a été ajoutée dans le service de gestion des articles afin de vérifier qu'un article est affecté à un seul type de structure : soit un laboratoire, soit un service organisationnel.

Résultat : les données enregistrées respectent désormais la règle métier définie et les affectations incohérentes sont refusées par l'application.

Gestion des statuts de campagne d'inventaire

Problème : le statut d'une campagne d'inventaire devait être déterminé automatiquement en fonction des dates de début et de fin ainsi que de l'état d'avancement de l'inventaire.

Cause : plusieurs situations devaient être prises en compte, notamment les campagnes à venir, en cours, terminées ou expirées.

Solution : une logique de calcul a été mise en place dans le backend afin de déterminer automatiquement le statut de chaque campagne à partir des dates enregistrées et du nombre d'articles inventoriés.

Résultat : le statut affiché dans l'application correspond désormais à la situation réelle de la campagne et permet de suivre plus facilement son avancement.

Problèmes de synchronisation entre le frontend et le backend

Problème : certaines fonctionnalités du frontend ne fonctionnaient pas correctement lorsque les données ou les structures attendues par le frontend ne correspondaient pas exactement aux réponses fournies par l'API backend.

Cause : le développement du frontend et du backend a nécessité plusieurs ajustements concernant les noms des champs, les formats des données et les endpoints utilisés.

Solution : les requêtes envoyées par le frontend et les réponses retournées par le backend ont été vérifiées à l'aide de Swagger et de tests API, puis les données ont été ajustées afin de respecter la structure définie.

Résultat : la communication entre le frontend Next.js et le backend Spring Boot a été stabilisée et les différentes fonctionnalités ont pu être intégrées progressivement.

Difficultés liées à l'affichage des informations de l'utilisateur connecté

Problème : le nom de l'utilisateur connecté ne s'affichait pas correctement dans certaines interfaces, notamment dans la barre de navigation de l'espace administrateur.

Cause : les informations de l'utilisateur connecté étaient récupérées à partir des données d'authentification, mais leur récupération ou leur utilisation dans le composant frontend nécessitait certains ajustements.

Solution : la récupération des informations de l'utilisateur connecté a été vérifiée et adaptée dans le service d'authentification ainsi que dans les composants de navigation concernés.

Résultat : les informations de l'utilisateur connecté peuvent désormais être récupérées et affichées correctement dans l'interface.

Erreurs de compilation lors de l'amélioration du PDF

Problème : lors de l'amélioration de la mise en page du document PDF, des erreurs de compilation liées à l'utilisation de la classe Color sont apparues.

Cause : l'erreur était liée à l'utilisation de la classe Color dans le code de génération du PDF et à la nécessité d'utiliser correctement la classe correspondant à la bibliothèque utilisée pour la génération du document.

Solution : les imports et l'utilisation de la classe Color ont été corrigés afin d'utiliser correctement la classe adaptée au contexte du projet et à la bibliothèque OpenPDF.

Résultat : la génération du document PDF a de nouveau fonctionné correctement, avec une mise en page améliorée et une présentation plus adaptée au rapport d'inventaire.

Contrôle des informations générées par l'intelligence artificielle

Problème : la génération du rapport textuel assisté par IA nécessitait de s'assurer que le contenu produit ne contienne pas d'informations inventées ou non vérifiées.

Cause : les modèles de langage peuvent, par nature, produire des formulations qui s'éloignent des données fournies si le contexte transmis n'est pas suffisamment précis ou contrôlé.

Solution : une attention particulière a été portée à la préparation des données transmises au modèle, afin que la génération du texte reste strictement basée sur les statistiques réellement calculées par l'application, notamment le nombre total d'articles, leur état et la progression de l'inventaire.

Résultat : le rapport généré reflète les données réelles du système. L'intelligence artificielle est ainsi utilisée principalement pour formuler et organiser une synthèse à partir des données disponibles, et non pour produire de nouvelles informations.

Gestion des anciennes données lors de l'évolution du modèle

Problème : certaines modifications apportées au modèle de données et aux énumérations ont entraîné des incompatibilités avec les données déjà présentes dans la base de données.

Cause : l'évolution du modèle au cours du développement a nécessité de modifier certaines valeurs et certaines règles métier alors que des données de test existaient déjà en base.

Solution : les données de test concernées ont été vérifiées et adaptées aux nouvelles règles du modèle. Des tests ont également été effectués après chaque modification afin de vérifier que les fonctionnalités existantes continuaient à fonctionner correctement.

Résultat : la base de données et le modèle applicatif sont devenus cohérents, permettant de poursuivre le développement avec une structure de données stable.

4.21 Bilan du projet

Résultats obtenus

L'application développée permet de centraliser les informations relatives au matériel, de gérer les catégories, les laboratoires, les services organisationnels et les utilisateurs, de suivre les mouvements en appliquant des règles métier côté serveur, de gérer les campagnes d'inventaire l'export des données invente en Excel et le pointage des articles, de sécuriser l'accès via une authentification JWT, de documenter et tester l'API avec Swagger et Postman, de produire des statistiques sur le patrimoine, et de générer un rapport d'inventaire assisté par intelligence artificielle exportable en PDF.

Ces résultats couvrent l'ensemble du périmètre fonctionnel identifié lors de la phase de spécification des besoins, aussi bien pour les fonctionnalités de gestion courante (articles, catégories, laboratoires, services, utilisateurs) que pour les fonctionnalités plus avancées, comme la sécurisation par JWT ou la génération de rapport assistée par intelligence artificielle. La cohérence entre les cas d'utilisation identifiés au chapitre 2 et les fonctionnalités effectivement réalisées a été vérifiée tout au long du développement, notamment à travers les tests présentés en section 4.19.

Compétences techniques acquises

Ce stage a permis de développer plusieurs compétences techniques concrètes : le développement backend en Java 21 avec Spring Boot selon une architecture en couches, la mise en pratique de Spring Data JPA pour la persistance des données, la conception de DTO, la mise en place d'une authentification JWT avec Spring Security, la documentation et le test d'API avec Swagger/OpenAPI et Postman, ainsi que le développement frontend avec Next.js, React, TypeScript et Tailwind CSS. La génération de rapport assistée par IA avec Ollama et la génération de documents PDF avec OpenPDF ont permis d'aborder des aspects plus spécifiques, à la croisée du développement classique et de l'intégration de modèles d'intelligence artificielle dans une application métier.

Compétences méthodologiques acquises

Au-delà des compétences techniques, ce stage a permis de mieux analyser un besoin avant de commencer le développement, et de le modéliser à travers des diagrammes UML plutôt que d'entrer directement dans le code. L'importance d'appliquer les règles métier et les contrôles de sécurité côté serveur est également apparue clairement. Enfin, Git et GitHub ont été utilisés pour suivre l'évolution du code et conserver un historique du travail réalisé.

Limites actuelles du projet

Malgré les fonctionnalités développées, certaines limites subsistent dans la version actuelle de l'application. Le choix d'un inventaire directement par article n'est pas encore totalement adapté au fonctionnement souhaité, notamment lorsqu'un grand nombre d'articles doit être inventorié.

De plus, lorsqu'un article est mis en **sortie définitive**, il doit être supprimé de la liste des articles actifs tout en restant conservé dans l'historique des mouvements afin de garder une trace de son parcours et d'assurer sa traçabilité.

La gestion de certaines opérations d'inventaire reste également dépendante de la saisie et de la validation manuelles des informations par les utilisateurs. Enfin, l'application ne dispose pas encore d'un système permettant aux utilisateurs de communiquer directement entre eux pour demander des informations ou signaler une situation concernant un article ou une opération d'inventaire.

Perspectives d'amélioration

Plusieurs pistes d'amélioration peuvent être envisagées pour la suite du projet, notamment l'enrichissement du tableau de bord avec des indicateurs supplémentaires, l'amélioration continue de la qualité du texte généré par l'intelligence artificielle et le renforcement des tests automatisés au-delà des tests manuels réalisés avec Postman. Une évolution possible consisterait également à historiser les rapports générés, afin de pouvoir comparer l'état du patrimoine d'une campagne d'inventaire à l'autre.

La mise en place d'une architecture avec **Docker** et Docker Compose pourrait également faciliter l'installation et le déploiement de l'application, notamment en regroupant le frontend, le backend et la base de données. Une mise en production sur un serveur ou une plateforme cloud, accompagnée éventuellement d'une intégration continue avec **GitHub** Actions, permettrait d'améliorer la maintenance et le déploiement de l'application.

Par ailleurs, un système de messagerie interne pourrait être ajouté afin de permettre aux responsables et aux agents de poser des questions ou de demander des informations concernant un article, une campagne d'inventaire ou l'utilisation de l'application. Un système de notifications pourrait également informer les utilisateurs des différentes actions importantes, telles que les nouveaux messages, les mouvements d'articles, les demandes ou les changements liés aux campagnes d'inventaire.

Une autre amélioration possible serait d'utiliser **JasperReports** pour développer une génération de rapports PDF plus avancée et personnalisable. Cela permettrait de créer des documents structurés à partir des données de l'application, notamment pour les campagnes d'inventaire, les listes d'articles et les rapports destinés à l'administration.

Conclusion du chapitre

Ce chapitre a présenté la réalisation complète du système, des choix techniques jusqu'aux tests, en passant par les fonctionnalités les plus avancées du projet : sécurité JWT, documentation Swagger, génération de rapport assistée par IA et export PDF. Les difficultés rencontrées en cours de route, présentées avec leurs solutions, ont constitué des occasions concrètes d'apprentissage.

Conclusion générale

Ce rapport a présenté le travail réalisé pendant un stage de deux mois à l'INRH, pour le Centre CSZIAM, autour du développement d'une application web de gestion d'inventaire du patrimoine matériel.

Face au problème de la dispersion des informations et du manque de traçabilité du matériel, une application a été conçue et développée. Elle s'appuie sur un backend Spring Boot / MySQL organisé en couches et sécurisé par authentification JWT, ainsi qu'un frontend Next.js / React / TypeScript, et permet de centraliser la gestion du matériel, des catégories, des laboratoires, des services organisationnels et des utilisateurs, tout en assurant le suivi des mouvements et des campagnes d'inventaire grâce à des règles métier appliquées côté serveur.

Le projet va au-delà d'une simple gestion de données : il intègre une documentation et un test complets des API avec Swagger/OpenAPI et Postman, une génération de statistiques sur le patrimoine, ainsi qu'une génération de rapport d'inventaire assistée par intelligence artificielle à l'aide d'un modèle local exécuté avec Ollama, exportable sous forme de document PDF structuré grâce à la bibliothèque OpenPDF.

Ce stage a été l'occasion d'approfondir des compétences techniques variées — Spring Boot, Spring Security, JWT, Spring Data JPA, DTO, Next.js, API REST, génération de PDF — mais aussi des compétences méthodologiques, comme l'analyse des besoins, la modélisation UML, ou encore les tests avec Postman et Swagger. Les difficultés rencontrées, notamment autour de la configuration de la sécurité, de la cohérence des données et de la génération du rapport, ont été surmontées progressivement et sont détaillées dans ce rapport avec les solutions apportées.

Le projet répond aux objectifs fixés au départ, en offrant au Centre CSZIAM une solution centralisée, sécurisée et traçable pour la gestion de son patrimoine matériel, tout en ouvrant des perspectives d'amélioration pour la suite, notamment sur l'enrichissement du tableau de bord et sur la qualité du texte généré par l'intelligence artificielle.

Bibliographie / Webographie

Cette section liste les ressources techniques officielles utilisées comme référence pendant le projet. Elle est à compléter avec les ressources réellement consultées au fil du stage.

Spring. Spring Framework Documentation. Documentation générale du framework Spring.
<https://docs.spring.io/spring-framework/reference/>

Spring. Spring Boot Reference Documentation. Documentation utilisée pour la mise en place du backend de l'application.
<https://docs.spring.io/spring-boot/index.html>

Spring. Spring Data JPA Reference Documentation. Documentation relative à l'accès aux données et à la gestion des entités JPA.
<https://docs.spring.io/spring-data/jpa/reference/>

Spring. Spring Security Reference Documentation. Documentation relative à la sécurité, l'authentification et l'autorisation.
<https://docs.spring.io/spring-security/reference/>

Spring. Spring REST Documentation. Documentation concernant la création de services REST avec Spring.
<https://spring.io/guides/gs/rest-service/>

Hibernate. Hibernate ORM Documentation. Documentation concernant la persistance des données avec Hibernate/JPA.
<https://hibernate.org/orm/documentation/>

Jakarta EE. Jakarta Persistence Specification. Référence concernant les entités et les relations JPA.
<https://jakarta.ee/specifications/persistence/>

Jakarta EE. Jakarta Bean Validation Specification. Référence concernant la validation des données dans les applications Java.
<https://jakarta.ee/specifications/bean-validation/>

Project Lombok. Lombok Documentation. Documentation concernant l'utilisation de Lombok dans les classes Java.
<https://projectlombok.org/>

Apache Maven. Maven Documentation. Documentation concernant la gestion des dépendances et la construction du projet Java.
<https://maven.apache.org/guides/>

Oracle. Java Documentation. Documentation officielle du langage Java et de la plateforme Java.
<https://docs.oracle.com/en/java/>

Oracle. MySQL Reference Manual. Documentation officielle du système de gestion de base de données MySQL.
<https://dev.mysql.com/doc/>

Swagger. OpenAPI Specification. Référence concernant la documentation et la description des API REST.
<https://swagger.io/specification/>

Springdoc. Springdoc OpenAPI Documentation. Documentation utilisée pour intégrer OpenAPI et Swagger UI avec Spring Boot.
<https://springdoc.org/>

Postman. Postman Learning Center. Documentation utilisée pour les tests et la vérification des API REST.
<https://learning.postman.com/>

JSON Web Token. RFC 7519 – JSON Web Token (JWT). Référence concernant le fonctionnement des jetons JWT utilisés pour l'authentification.
<https://datatracker.ietf.org/doc/html/rfc7519>

OWASP Foundation. OWASP Top 10. Référence concernant les principaux risques de sécurité des applications web.
<https://owasp.org/www-project-top-ten/>

OWASP Foundation. OWASP Cheat Sheet Series. Guides de bonnes pratiques concernant la sécurité des applications web.
<https://cheatsheetseries.owasp.org/>

Next.js. Next.js Documentation. Documentation officielle du framework utilisé pour le développement du frontend.
<https://nextjs.org/docs>

React. React Documentation. Documentation officielle de la bibliothèque utilisée pour la construction des interfaces.
<https://react.dev/>

TypeScript. TypeScript Documentation. Documentation du langage utilisé pour le développement frontend lorsque TypeScript est utilisé.
<https://www.typescriptlang.org/docs/>

MDN Web Docs. JavaScript Documentation. Référence concernant JavaScript, HTML, CSS et les technologies du web.
<https://developer.mozilla.org/>

Git. Git Documentation. Documentation officielle du système de gestion de versions utilisé pendant le développement.
<https://git-scm.com/doc>

GitHub. GitHub Documentation. Documentation concernant la gestion des dépôts et du code source sur GitHub.

<https://docs.github.com/>

Ollama. Documentation officielle. Documentation concernant l'exécution et l'utilisation de modèles d'intelligence artificielle en local.

<https://docs.ollama.com/>

OpenRouter. Documentation API. Documentation concernant l'utilisation d'API permettant l'accès à différents modèles d'intelligence artificielle.

<https://openrouter.ai/docs>

OpenPDF. Documentation du projet. Documentation concernant la génération de documents PDF en Java.

<https://github.com/LibrePDF/OpenPDF>

Figma .créations des maquettes pour les interfaces frontend

<https://www.figma.com/files/team/1577249513760900214/recents-and-sharing?fuid=1577249511165052565>

Annexes

Annexe A — Diagrammes UML complémentaires

Les diagrammes de séquence relatifs à la création d'un mouvement et à l'authentification JWT sont présentés au chapitre 3 (section 3.8). Cette annexe reste disponible pour tout autre diagramme UML complémentaire jugé utile.

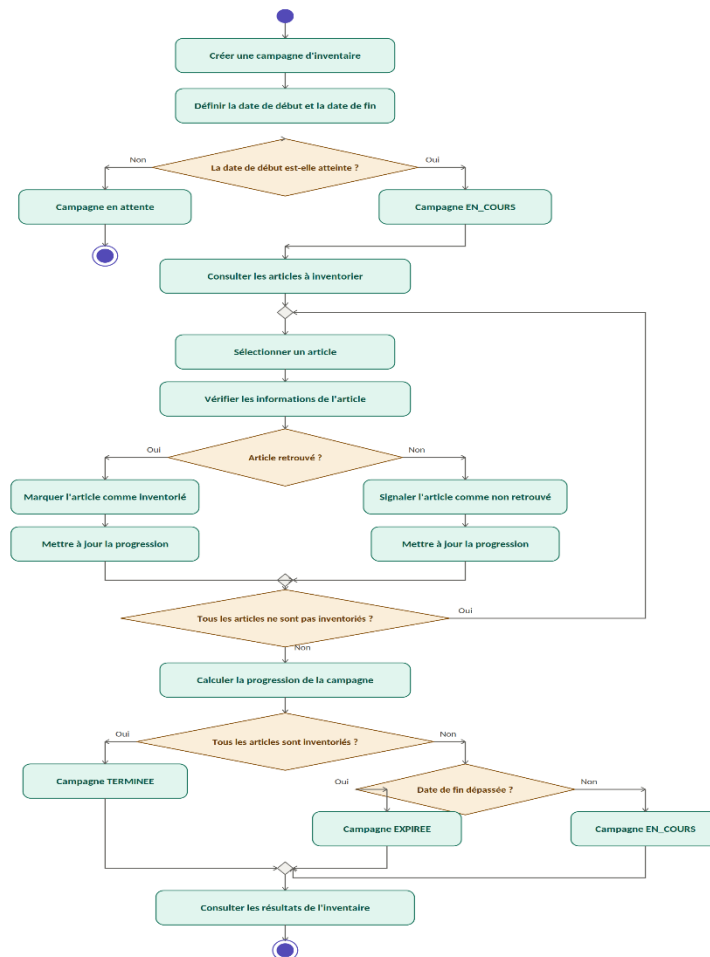


Figure 24-Diagramme activité campagne inventaire

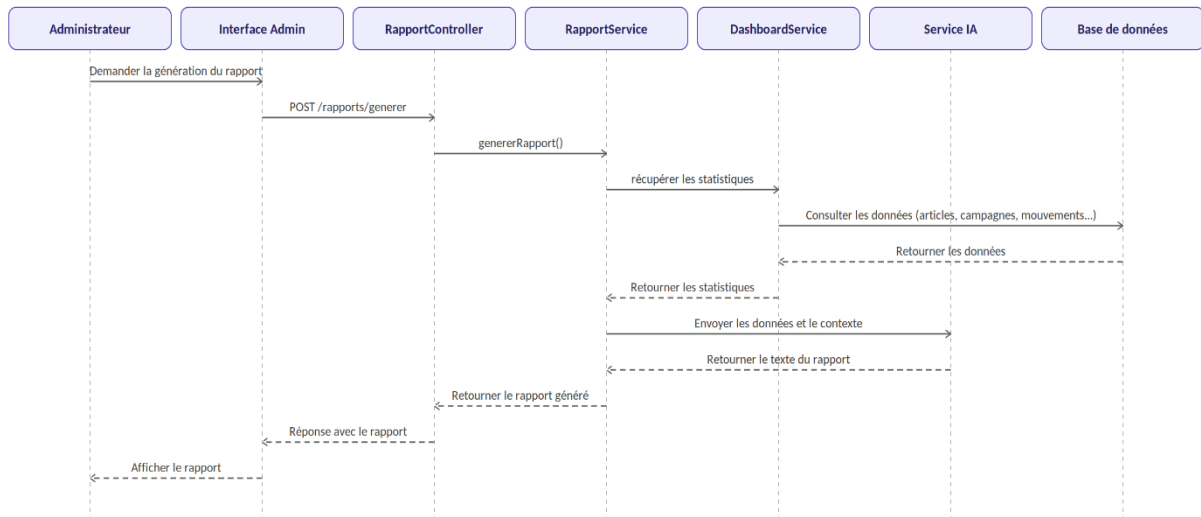


Figure 25-Diagramme séquence générations rapport IA

Annexe B — Captures Swagger complémentaires

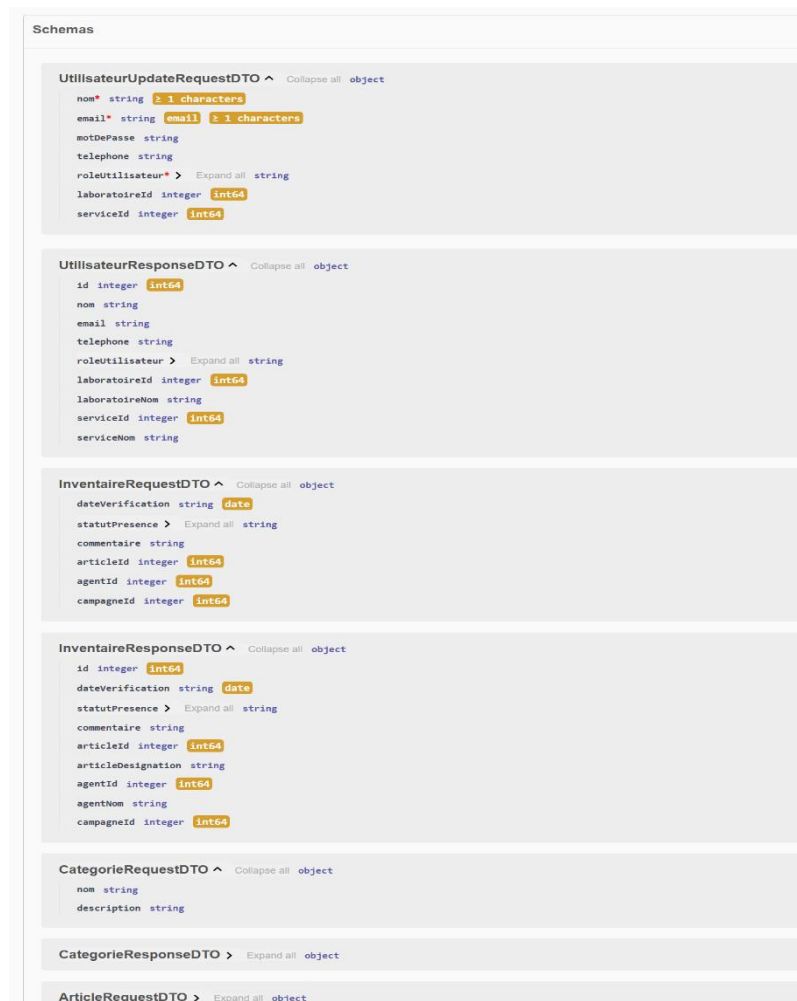


Figure 26-Structure des tables principales

Annexe C — Autres documents techniques

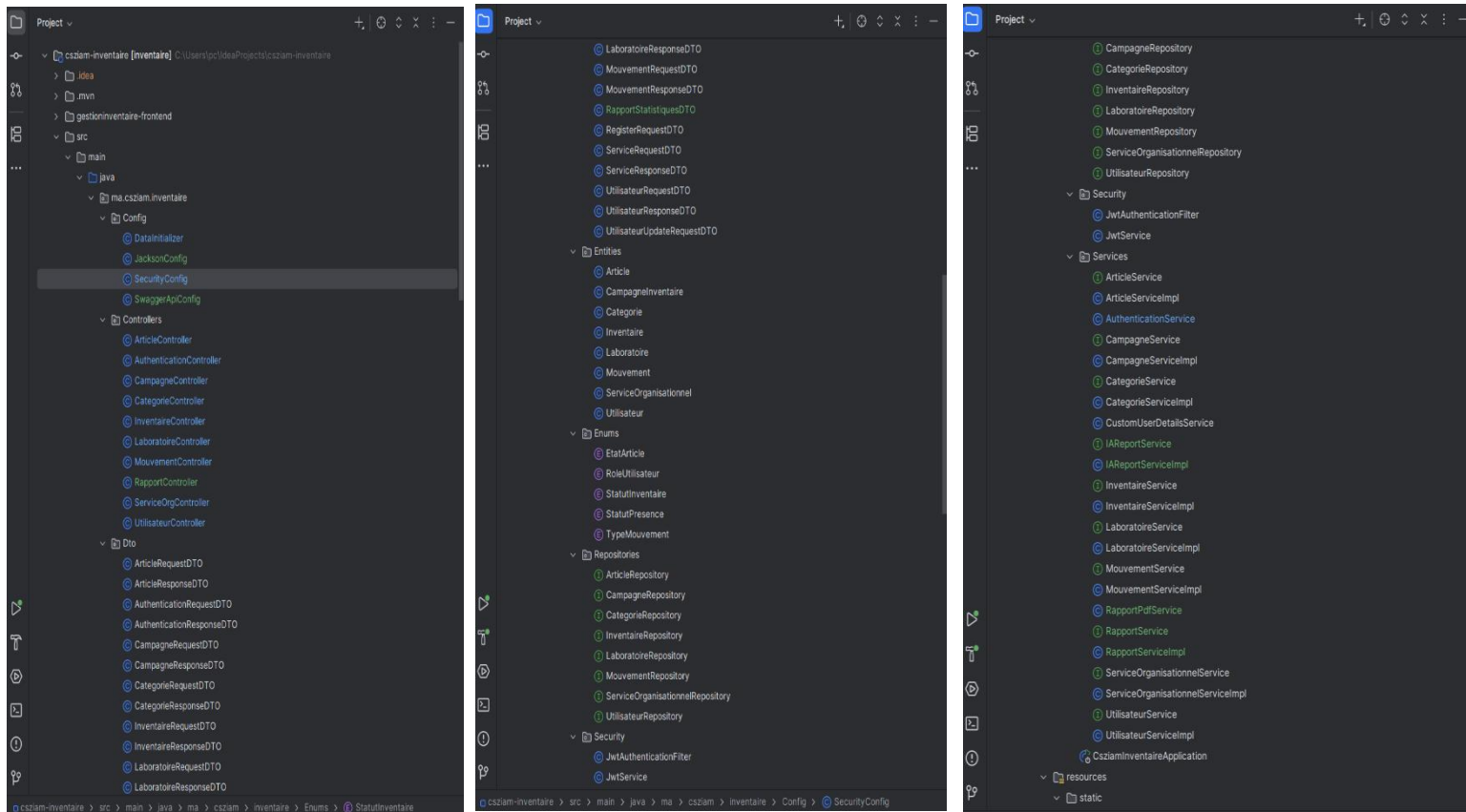


Figure 27-Architecture en couches de l'application Spring Boot

```
public class IAReportServiceImpl implements IAReportService {
    private String construirePrompt(RapportStatistiquesDTO s) {
        prompt.append("""
        Tu es un assistant spécialisé dans la rédaction de rapports administratifs professionnels pour la gestion du patrimoine et des inventaires.

        À partir UNIQUEMENT des données fournies ci-dessous, rédige un rapport d'inventaire professionnel en français.

        RÈGLES ABSOLUES :

        - Utilise uniquement les données fournies.
        - N'invente aucune donnée, aucun événement, aucune cause et aucune information.
        - Ne déduis aucune cause à partir des statistiques.
        - Ne présente jamais une situation comme bonne, mauvaise, importante, élevée, faible, insuffisante, satisfaisante ou préoccupante sans comparaison objective fournie dans les données.
        - Ne considère jamais "en panne", "en maintenance" ou "hors service" comme des catégories. Ce sont des états des articles.
        - Présente tous les types de mouvements fournis dans les données.
        - Ne considère pas automatiquement un nombre élevé de mouvements comme un problème.
        - Ne prétends jamais qu'une action a déjà été réalisée si cela n'est pas indiqué dans les données.
        - Les recommandations doivent être formulées comme des actions futures ou des mesures de suivi.
        - Chaque recommandation doit être directement liée à une donnée ou à un constat du rapport.
        - Les pourcentages doivent être calculés uniquement à partir des nombres fournis.
        - Si une donnée est absente, ne l'invente pas.
        - Utilise un style professionnel, administratif, clair, neutre et objectif.
        - Présente les statistiques principalement sous forme de paragraphes.
        - Utilise des listes numérotées uniquement dans les sections "PRINCIPAUX CONSTATS" et "RECOMMANDATIONS".
        - Les titres des sections doivent être clairement séparés du contenu.
        - Utilise la notation française pour les nombres et les pourcentages : espace pour les milliers et virgule pour les décimales.
        - Écris "268 400 DH" et non "268400.0 DH".
        - Écris "0 %" et non "0.0 %".

        STRUCTURE OBLIGATOIRE :

        INTRODUCTION

        Présente brièvement l'objectif du rapport et indique qu'il présente les données relatives au patrimoine du Centre CSZIAM pour l'année concernée.

        SYNTHÈSE GÉNÉRALE
        """)
    }
}
```

Figure 28-Extrait du prompt ou du contexte envoyé à Ollama

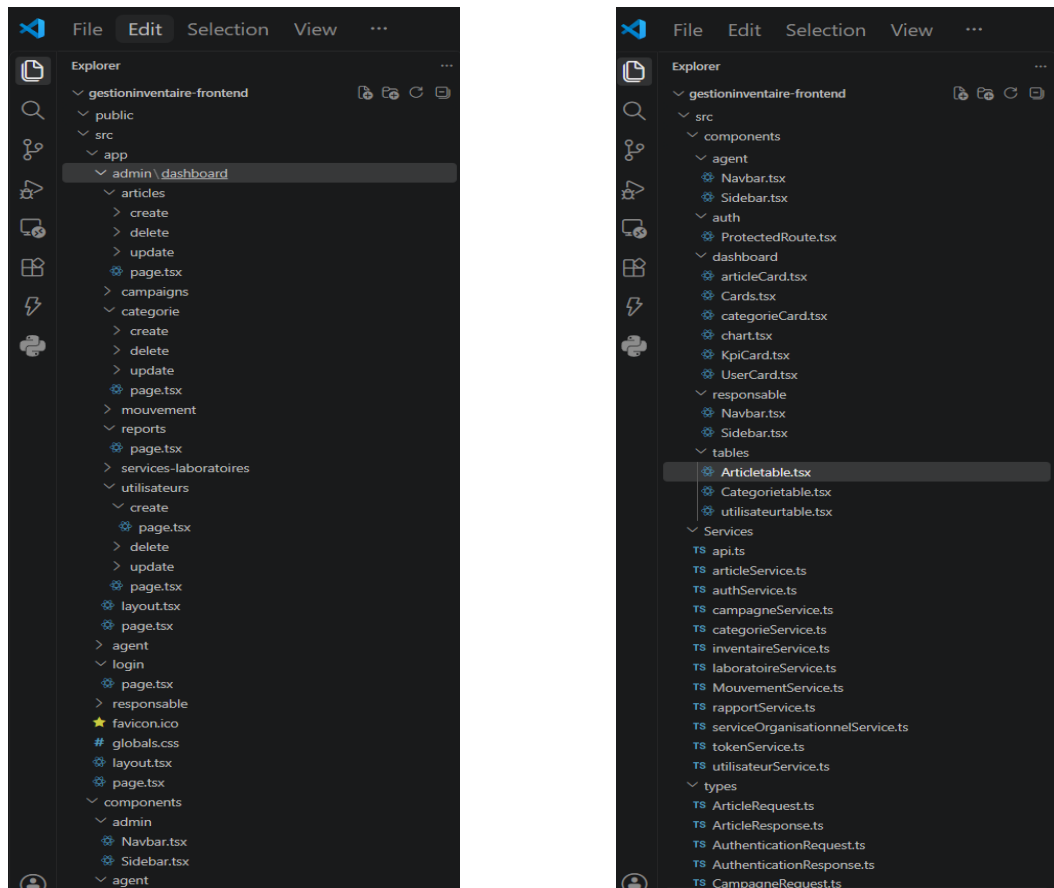


Figure 30-Architecture de l'application Next.js

```

32 public class SecurityConfig {
33
34     @Bean
35     public SecurityFilterChain securityFilterChain(HttpSecurity http)
36         throws Exception {
37
38         http
39
40             .csrf((CsrfConfigurer<HttpSecurity> csrf) -> csrf.disable())
41
42             .cors((CorsConfigurer<HttpSecurity> cors) -> {})
43
44             .sessionManagement((SessionManagementConfigurer<HttpSecurity> session) ->
45                 session.sessionCreationPolicy(SessionCreationPolicy.STATELESS)
46             )
47
48             .authorizeHttpRequests((AuthorizationManagerRequestMat... auth) -> auth
49                 .requestMatchers(@"/auth/login").permitAll()
50                 // Swagger / OpenAPI
51                 .requestMatchers(
52                     @"/swagger-ui.html",
53                     @"/swagger-ui/**",
54                     @"/v3/api-docs",
55                     @"/v3/api-docs/**",
56                     @"/swagger-resources/**",
57                     @"/webjars/**"
58                 ).permitAll()
59                 .requestMatchers(@"/auth/register").hasRole("ADMIN")
60                 .anyRequest().authenticated()
61             )
62
63             .addFilterBefore(
64                 jwtAuthenticationFilter,
65                 UsernamePasswordAuthenticationFilter.class
66             );
67
68         return http.build();
69     }
70 }

```

Figure 29-Extrait de la configuration SecurityConfig